ENS 491-492 – Graduation Project

Final Report

Project Title:

Analysis, Implementation and Control of a Parallel Spherical Manipulator

Group Members:

Ömür Kaan Sarı 30561

Göktuğ Arda Gök 30975

Supervisor(s):

Volkan Patoğlu

Date:

01.04.2026



1. EXECUTIVE SUMMARY

This graduation project focuses on the development and improvement of a Spherical Parallel Robot, a system designed to teach students complex robotics concepts like 3D rotations and Quaternion mathematics. The robot is "holonomic," meaning it can rotate in any direction without mechanical limits. While the mechanical structure was built by previous groups, this project specifically addresses the system's most critical failure: position drift.

In the previous design, the robot estimated its position using only data from the motor encoders (sensors that count wheel turns). This method relies on the assumption that the wheels never slip against the sphere. However, in reality, factors like sudden acceleration, variable friction, and mechanical flexibility cause micro-slips. Because the system calculates position by adding up these movements over time (integration), even tiny errors accumulate. This causes "drift," where the physical orientation of the robot no longer matches what the computer calculates.

To solve this, our group introduced a Vision-Based Drift Compensation system. Initially, we investigated optical flow sensors (like computer mice), but experiments proved they were unreliable due to the sphere's curvature and focus issues. Instead, we developed a novel camera-based system using a Raspberry Pi and a wide-angle camera. We applied AprilTag markers to the sphere's surface and implemented a mathematical algorithm called Ray-Sphere Intersection. This allows the system to calculate the sphere’s absolute 3D orientation geometrically, rather than guessing it from wheel turns.

The final system integrates this visual data into the main control loop (Simulink) using UDP communication. By comparing the seen position from the camera against the calculated position from the encoders, the system can now detect and measure drift in real-time. This project successfully bridges the gap between a theoretical mechanical design and a reliable educational tool by giving the robot the ability to observe and correct its own movement.

2. PROBLEM STATEMENT

The parallel spherical manipulator project is a novel entry as an educational spherical haptic device and due to it being novel, no exact or even remotely close research being available, many problems arise during both the theoretical and experimental stages of the project. Robotics field and rotations by itself are areas that not many educational purposed work with the main motive of educating about the field.

Our motivation is to overcome the limitations previously addressed in literature about traditional parallel spherical manipulators and to put the project in a form of educational device to eventually solve the parallel manipulator problem globally.

The robot of interest in this project is a three DOF system. The hardware setup consists of the motor drivers, fans and circuit components that are at the bottom, the steering and driving motors that are connected to the two wheels that are responsible of the spheres motion, the sphere itself that is stabilized in its place by bearings and the optical camera that is placed right above the sphere to monitor the orientation and rotation of the ball. The above mentioned bearings ensure there is no translation and the motion is limited to rotation only.

Multiple problems of the project can be counted as, kinematic analysis, real time control algorithm design, drift between wheels and the sphere and the sensor implementation problem to compensate for that.

Us being the third overall group to work on the project, some of the listed problems were already addressed before us; therefore, our main objective was to perfect the hardware setup and the control unit and most importantly, to implement the sensor system.

The reason why drift compensation with the sensor implementation is crucial is that the wheels tend to fail performing the perfect tracking of the reference signals they receive. The problem that is meant by calling it the 'Drift problem' actually consists of the lack of contact that occurs between the wheels and the sphere at some specific orientations of the system and momentarily occurring unnecessarily high contact/friction between the wheels and the sphere. These in combination cause the wheels to fail tracking the reference and/or cause the sphere to do irrelevant rotations.

The sensor that needs to be implemented should monitor the drift from the intended rotation and put this error into the closed loop controller of the motors to increase the precision of the sphere rotation.

2.1. Objectives/Tasks

The first objective of the project that had been addressed by the previous project members' efforts was to derive the kinematic and dynamic analysis of the system. The theoretical calculations in this step enables the realization of the project.

The second objective was obtaining an operating system. As the hardware was already put together before, we worked on labelling and organising the setup. All wires and components (motors, drivers etc) were labelled and put into order with correct labelling identifying them. This process was crucial as regardless of how perfect the control algorithm is or how consistent the sensor system is on monitoring the rotations, we would not be able to obtain the results we seeked if the hardware setup had flaws.

The third objective was developing a control algorithm to enable us to have real time consistent control over the rotations of the hardware setup. Once again, a Simulink block diagram using MATLAB in coordination with Quanser (encoder system) was prepared by the previous students for individual

control over the 4 motors that drive the system (2 steering and 2 driving motors). PID control units were used for each of the motors with varying parameters that were optimized for the best results. Precise filtering was also made.

The main objective of our work in this project was to design and implement a sensor system capable of monitoring the real rotation of the sphere in order to compensate for the drift occurring between the driving wheels and the sphere. Previous project groups had attempted to solve this problem by using optical mouse sensors, which can measure changes in the x and y directions with high precision under ideal flat-surface conditions. However, after testing these sensors individually on the spherical surface, it was observed that they could not provide reliable or meaningful data due to surface curvature, sensitivity to distance, and alignment issues. As a result, this approach was abandoned.

Instead, a vision-based sensing system was developed using an optical camera. To enable reliable rotation detection, a unique surface pattern was designed and applied to the sphere in the form of ArUco markers with unique IDs. By tracking these markers from a fixed camera placed above the sphere and applying geometric calculations, the system is able to estimate the absolute orientation of the sphere in three-dimensional space. This approach allows the rotation to be measured independently of the wheel motion and is therefore not affected by slip or friction variations.

After implementing the vision-based sensor system, the existing control algorithm was extended to make use of this new information. The orientation measured by the camera is compared with the orientation estimated from the motor encoders, and the resulting difference is interpreted as drift. This drift information is then fed back into the control loop as an error signal. By continuously correcting this error during operation, the system reduces the difference between the desired orientation and the actual orientation of the sphere, leading to more accurate, stable, and reliable rotational behavior throughout the motion.

2.2. Realistic Constraints

Hardware complications were encountered throughout our work on the project which is a common complication in hardware oriented projects such as ours. Also due to the original setup being made several years ago, some components needed to be changed. There were minor electrical leakages across the motors mountings that caused unwanted operation of motors.

The repairs on the hardware were made during the first months of our time working on the project however, when hardware was being tested in compliance with the sensor system, more problems emerged especially with the SML. We had already anticipated a problem concerning the encoder of SML due to it failing to properly track references that SMR was tracking almost perfectly. The encoder was not sending any signals to the control algorithm and the controller tried to reach the reference by making the motor run at full speed at all times. After we found out that the cable had snapped at the connection point and changed it, the motor started to act as intended with proper tracking.

Even after this, SML operated with a noise that did not exist with the other motors and lost track of the reference occasionally. It seemed as if the reference was non-existent and the motor operated on random signals. It was later found out that the wheel itself was loose on the motor's shaft and it slipped on it. This caused the wheel to sometimes miss the rotations of the motor shaft and as the encoder is measuring the orientation of the wheel, this caused huge differences between the reference and the actual position. This led to huge input signals being delivered to the motor and therefore the noise that is caused by high operating speed and the so-called random rotation of the wheel.

The powering cable of the SML snapped as well after the repairs on the other components of SML. Soldering and taping was enough to fix it properly; however, all these frequent problems show the complications about conducting a hardware oriented project that is being worked on for many years. Also each component being connected to multiple other components via independent cables and the system itself sometimes doing sudden and fast motions caused problems concerning the wellbeing of these cables.

The development board we have used as the controlling computer for the optical camera, Raspberry Pi 5 had its own limitations on our work due to its 8 Gigabyte RAM. Our camera operated with around 17 frames per second while it had the ability to work with more than 100 fps.

Another limitation we have encountered was that the surface pattern we implemented to the sphere was vulnerable to high amounts of light exposure as the markers reflected the light and were seen as large white spots instead of an ArUCO marker with a specific ID. All ArUCO markers were covered with matte varnish to decrease the reflectivity of them. The 3rd layer on top of the system also acts as a shade for the sphere to remain not completely exposed to light.

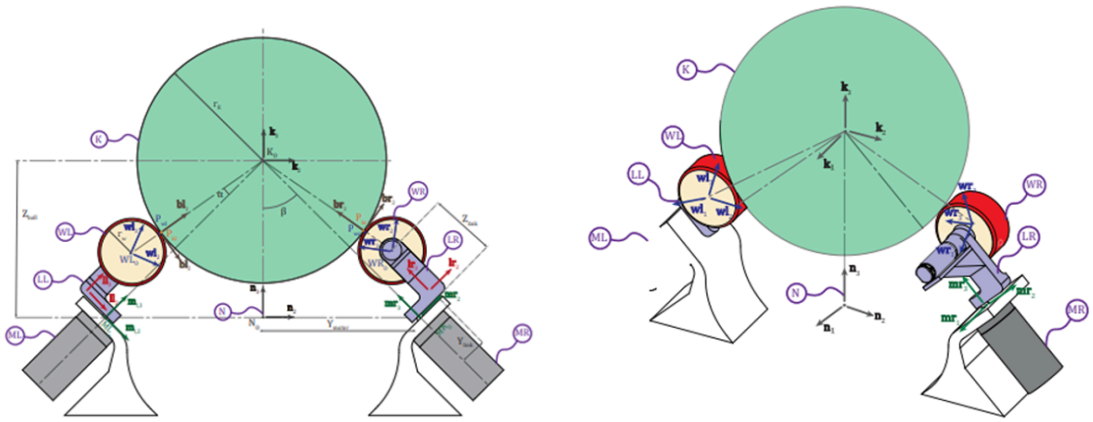
Economic constraints had their effects on our work. Before the mouse sensor idea was scrapped, replacing the old several years old sensors with new ones was our first alternative to the original sensors. However, due to such new optical sensors prices being quite high and that we needed at least 3 of such sensors with still no guarantee of them being implemented the way we presumed, the idea was scrapped.

Regarding sustainability, other than some components needing to be changed after their lifecycles are concluded either early or on time, the overall setup can be used for a long period of time due to it not containing any component that has to endure harsh conditions nor stresses.

3. METHODOLOGY

3.1 Kinematics

The kinematic and dynamic analysis made by the previous groups were analyzed and comprehended. Below can be seen the schematic for the system with every components representation for the kinematic model.



N representing the Newtonian reference frame, the robot mainly consists of five bodies thatare: the sphere K, right and left links LR and LL and right and left wheels WR and WL. Two additional auxiliary reference frames were also defined to represent the normal and tangential directions on the contact points of the wheels and the sphere, BR and BL. The points that are in the centers of the sphere, the newtonian reference frame and the wheels, (K0, N0, WR0 and WL0) represent the origins of the components respectively. The points MR0 and ML0 are to represent the centers of the axes of rotations of the right and left steering motors. The contact point of the sphere and the wheels are defined with Pkr & Pkl on the sphere surface and Pwr & Pwl on the wheel surfaces. Four angles were defined as well for the orientations of the steering motors and the wheels to be used on kinematic calculations: ψr, ψl for steering motors and φr, φl for the wheels.

This section explains the methods and techniques used to track the actual rotational motion of the spherical end-effector and to address the drift problem observed in the parallel spherical manipulator. The methodology followed an experimental and iterative approach, where multiple sensing solutions were implemented, tested, and evaluated based on their performance and practicality. Based on the results obtained during these experiments, the sensing strategy was revised and improved throughout the project.

3.1.1 Configuration Level Kinematics

The system's rotations are defined for the spherical robot's initial rotational motion. In relation to the chosen Newtonian frame, the sphere rotates. A series of consecutive simple rotations throughout multiple frames are used to derive this rotation. The steering motors' auxiliary reference frames, MR and ML, have undergone simple rotations with amounts of β and −β around the direction n1 axis, respectively. The amounts of φr and φl were used to spin body LR and LL around the mr3 and ml3 axes, respectively. Next, by performing simple rotations about the lr1 and ll1 axes with amounts of α and −α, respectively, the auxiliary frame BR and BL were aligned with the ball. These rotations can be expressed as:

$$R^{WR} = NR^{MR}_{\beta,n_1}\ MRR^{LR}_{\varphi_r,mr_3}\ LRR^{BR}_{\alpha,lr_1}\ BRR^{WR}_{\psi_r,br_1}$$

$$NR^{WL} = NR^{ML}_{-\beta,n_1}\ MLR^{LL}_{\varphi_r,ml_3}\ LLR^{BL}_{-\alpha,ll_1}\ BLR^{WL}_{\psi_r,bl_1}$$

The spherical rotation with regard to the inertial reference frame, N, where 3D rotations take place without representation singularities, is represented using unit quaternion representation. Using Euler parameters ($\dot{\epsilon}_1$, $\dot{\epsilon}_2$, $\dot{\epsilon}_3$, $\dot{\eta}$), the rotation of the sphere, K, is determined about the axis λ by an amount of θ. The Euler parameters are calculated using the equations below:

$$\eta = \cos cos\left(\frac{\theta}{2}\right)$$

$$\epsilon = \sin sin\left(\frac{\theta}{2}\right)$$

As they are necessary to establish the non-holonomic motion constraints, the contact points are the most significant position vectors. The following equation defines the location of the ball and wheel contact points on the ball side, Pkr and Pkl, from the origin of the Newtonian reference frame N0. In a similar manner, the Pwr and Pwl points' location vectors from the N0 point are also expressed.

$$r^{N_o P_{kr}} = r_K \times -br_3,\ r^{N_o P_{kl}} = r_K \times -bl_3$$

$$r^{N_o P_{wr}} = r_W \times br_3,\ r^{N_o P_{wl}} = r_W \times bl_3$$

3.1.2 Motion Level Kinematics

When representing the angular velocity of a rigid body, generalized speeds can be useful. To do this, the body's angular velocities are described in a reference frame defined by its generalized coordinates. The following equation, which provides a solution for the time rate of change of each generalized coordinate, determines the generalized speeds. Kinematical differential equations are the name given to these equations.

$$u_i = \sum_{s=1}^{n} Y_{i,s}(q1, \ldots, qn)\dot{q}_s + Z_i(q1, \ldots, qn), \qquad (i = 1, \ldots, n)$$

The kinematical DE's of our robot joints are defined as the time derivatives of their angles as it can be seen below:

$$u_4 = \dot{\psi}_r \qquad u_5 = \dot{\psi}_l \qquad u_6 = \dot{\phi}_r \qquad u_7 = \dot{\phi}_l$$

Due to the fact that we have used Euler parameters to define the spheres orientation, the kinematical DE of the sphere should be written using Euler parameters as well:

$$\dot{\epsilon}_1 = \frac{1}{2}(\epsilon_3\left(u_2\left(2{\epsilon_1}^2 + 2{\epsilon_3}^2 - 1\right) - 2\epsilon_2\epsilon_3 u_3\right) - \epsilon_2\left(u_3\left(2{\epsilon_1}^2 + 2{\epsilon_2}^2 - 1\right) - 2\epsilon_2\epsilon_3 u_2\right) + \eta(u_1 - 2\epsilon_2\eta u_3 + 2\epsilon$$

$$\dot{\epsilon}_2 = \frac{1}{2}(\epsilon_1\left(u_3\left(2{\epsilon_1}^2 + 2{\epsilon_2}^2 - 1\right) - 2\epsilon_1\epsilon_3 u_1\right) - \epsilon_3\left(u_1\left(2{\epsilon_2}^2 + 2{\epsilon_3}^2 - 1\right) - 2\epsilon_1\epsilon_3 u_3\right) + \eta(u_2 - 2\epsilon_1\eta u_3 + 2\epsilon$$

$$\dot{\epsilon}_3 = \frac{1}{2}(\epsilon_2\left(u_1\left(2{\epsilon_2}^2 + 2{\epsilon_3}^2 - 1\right) - 2\epsilon_1\epsilon_2 u_2\right) - \epsilon_1\left(u_2\left(2{\epsilon_1}^2 + 2{\epsilon_3}^2 - 1\right) - 2\epsilon_1\epsilon_2 u_1\right) + \eta(u_3 - 2\epsilon_1\eta u_2 + 2\epsilon_2\eta u_1$$

$$\dot{\eta} = -\frac{1}{2}\left(\epsilon_1 u_1 + \epsilon_2 u_2 + \epsilon_3 u_3\right)$$

Now that we have achieved the Kinematic DE's and the angular velocities, we can write the angular velocities of the sphere and the joints as it can be seen below:

$$N\omega^K = \left(u_1 n_1 + u_2 n_2 + u_3 n_3\right)$$

$$N\omega^{MR} = u_4 m r_3 \qquad N\omega^{ML} = u_5 m l_3$$

$$N\omega^{WR} = u_6 w r_3 \qquad N\omega^{WL} = u_7 w l_3$$

3.2 Drift Compensation Problem and Sensing Requirements

The spherical manipulator is driven by wheels that are in rolling contact with the sphere. Although the motor encoders provide precise information about the commanded motion of the wheels, they do not provide direct information about the actual motion of the sphere. Due to factors such as surface imperfections, wheel slip, contact force variations, and alignment errors, mechanical drift occurs between the wheels and the sphere.

As a result, the encoder-based estimation of sphere orientation gradually diverges from the true orientation. This drift becomes especially problematic during long-duration or continuous rotations, where even small errors accumulate over time.

To compensate for this drift, an external sensing method is required that satisfies the following requirements:

- Ability to observe the **actual surface motion** of the sphere
- Independence from motor encoders
- Capability of operating under **unlimited rotation**
- Sufficient resolution to detect small orientation changes
- Practical integration with the existing hardware setup

Based on these requirements, the sensing problem was approached as a **surface motion tracking** problem.

3.3 Control

Control over the mechanical system is done using MATLAB/Simulink and QUARC software. The motor controlling block diagram prepared in the Simulink environment combined with the values coming from the encoders and via the QUARC board, reference tracking is made possible.

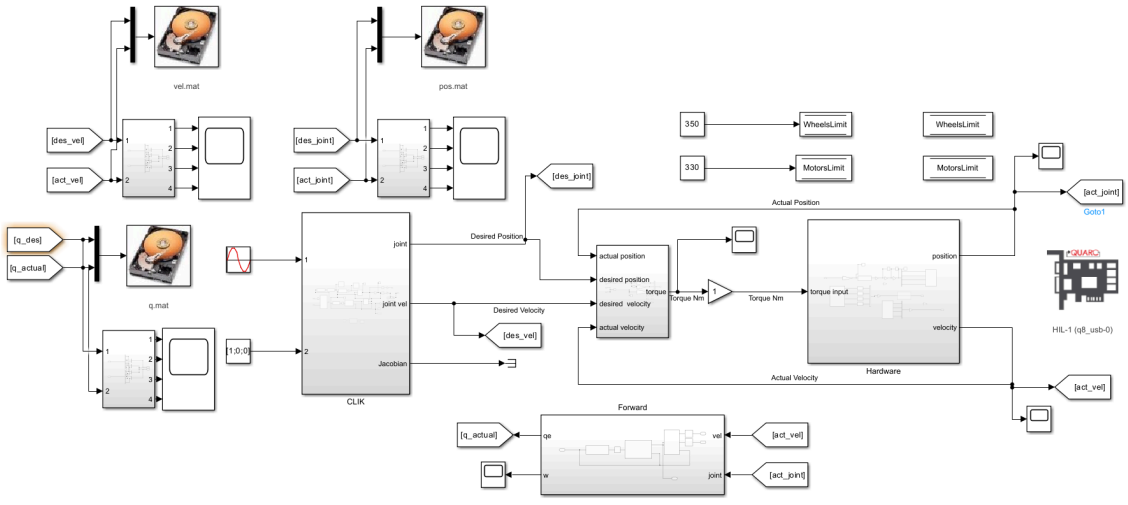


Figure 1. Complete control algorithm.

Overall, the control algorithm used by the previous group was used but enhanced in ways that were seen as beneficial. PID gains were updated and necessary adjustments were done on the initial values for the reference joint positions so that the sinusoids are centered around zero point.

3.3.1 Tuning

Each of the joints (motors they are controlled by) are tuned with respect to data from their datasheets. These values were used to convert the torque value, that is calculated using the desired and actual position and velocity values and the PID parameters, to voltage value that is going to be fed to the motors.

The torque values are first adjusted according to the gear ratios used in the mechanical system to obtain the desired values on the motor shafts. This value is then converted to current via specified ratios for each of the big and small motors (1/38.9 for big motors, 1/16.2 for small motors). At last, current is converted to voltage with the ratios for BMs 1/1.78 and Sms 1/2.

3.3.2 Closed Loop Inverse Kinematics

The CLIK algorithm implemented to our system takes the reference signal, in sinusoidal form in our case, and the desired axes that the user wants that signal to be executed and outputs the desired joint angles and velocities. Which are also fed back into the loop for further tracking throughout the motion.

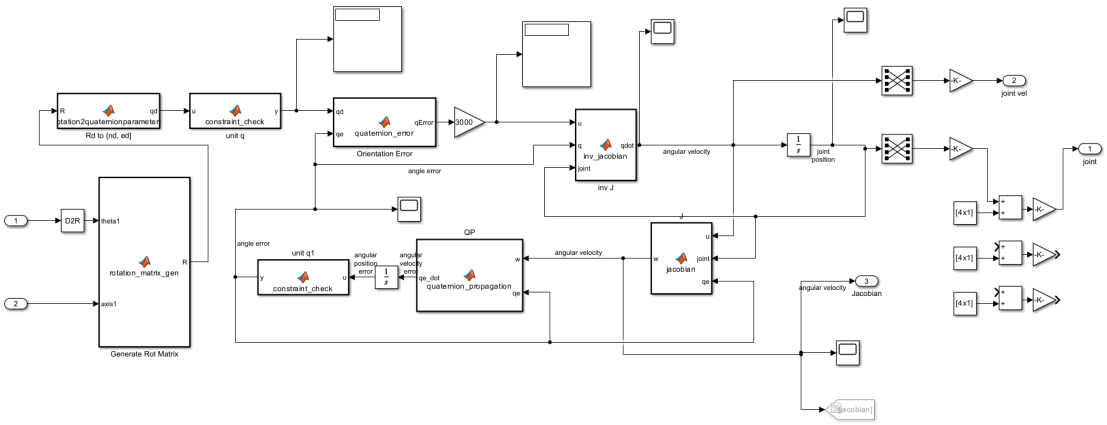


Figure 2. CLIK block.

3.3.3 PID Controllers

The desired orientation and velocity values along with the actual orientation and velocity values that are coming from the encoders are fed into the PID controller block that contains individual parameters for all four of the motors, that all were found as a conclusion of multiple test cases.

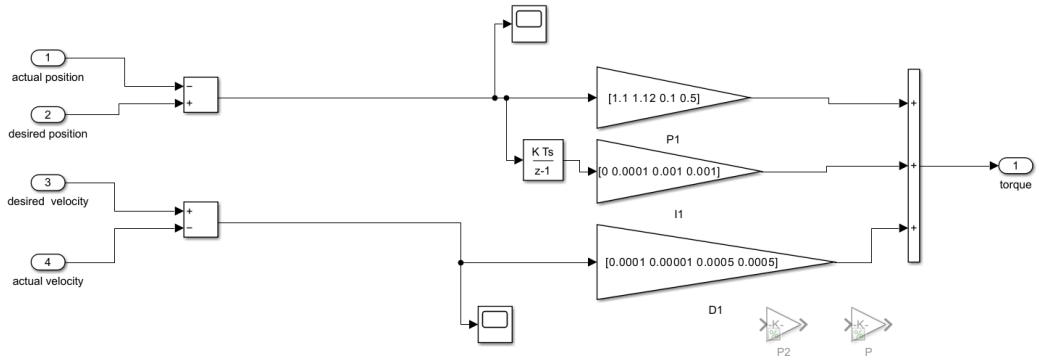


Figure 3. PID control block.

BML is the only motor to use a PD controller while all other three motors are using an integrator gain as well. The parameters for the gains were first tested on the single motor simulations and then were also tried with all four motors running simultaneously, executing a rotation on the sphere. Even though the exact values of these gains changed, the ratios between them remained similar to what we saw in single motor tests.

Being the exact same motors with the exact same drivers, SML and SMR did not perform similarly when the same PID values were used. SMR showed almost perfect tracking with a Kp value that is 4 times larger than the one of SML. This shows the imperfections in real life situations and also some currently unknown alterations that are caused by both operating near its limit on certain situations and a long operating period.

3.3.4 Hardware Block

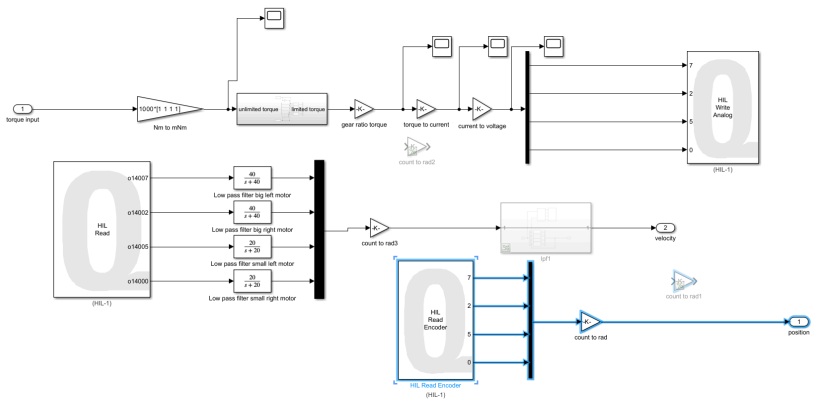


Figure 4. Hardware block.

The required torque values for each motor that is calculated after the PID blocks are first converted into mNm from Nm. Then they are all fed into saturation blocks with limits set by the data taken from motor data sheets so that none of the motors exceed the safe zone of operation.

Torque is then converted to current and voltage to be fed to motors as already explained in section 3.3.1 Tuning.

Encoder readings are converted to radians from encoder counts in this block as well.

Communication between the control algorithm and the encoders/motors are done using specific QUARC blocks that exist in the Simulink environment with the QUARC license.

3.4 Optical Mouse Sensor–Based Sensing Approach

3.4.1 Motivation and Principle

The first sensing method investigated was based on optical mouse sensors. Optical mouse sensors internally estimate motion by capturing consecutive images of a surface and computing optical flow. These sensors output incremental motion values in the x and y directions at high update rates.

The motivation for using optical mouse sensors was that they directly measure surface displacement and are widely used in precision motion tracking applications on flat surfaces. The initial idea was to adapt this technology to the spherical surface by placing multiple sensors around the sphere.

In theory, by placing three optical sensors approximately 120 degrees apart, it would be possible to observe surface motion in multiple directions and reconstruct the rotational motion of the sphere.

3.4.2 Hardware Setup

The optical sensors used during the experiments were ADNS-2610 optical mouse sensors. Each sensor was tested independently before attempting multi-sensor integration. An Arduino Nano was used as the microcontroller with manual wiring for communication. Special

care was taken to maintain the recommended lens-to-surface distance specified in the datasheet (approximately 2–2.5 mm).

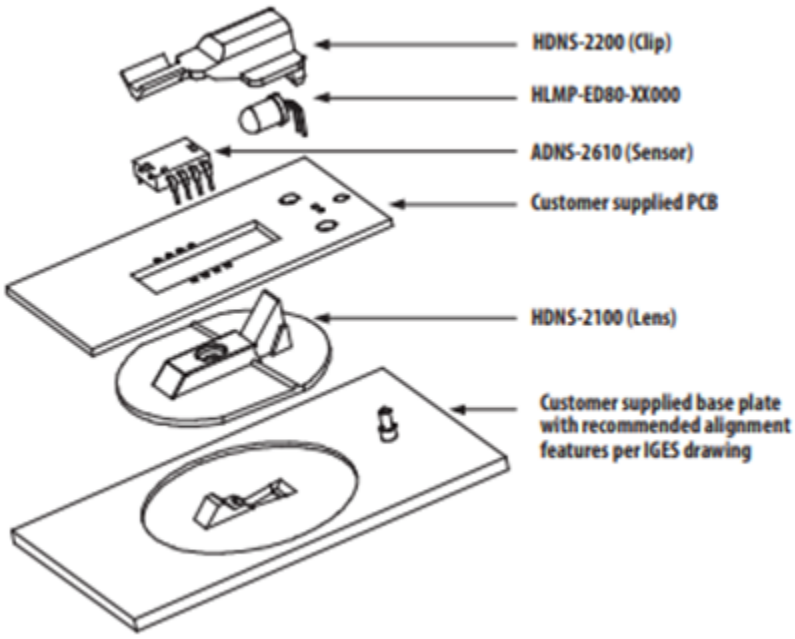


Figure 5. Optical mouse sensor schematic.

3.4.3 Experimental Observations and Limitations

Despite extensive experiments with multiple configurations, the sensor outputs were found to be inconsistent. Several fundamental limitations were identified:

1. Surface Curvature: Optical mouse sensors are designed for flat surfaces. The curved geometry of the sphere significantly degrades their ability to track motion reliably.

2. Strict Distance Requirements: The sensors require a very precise distance from the surface. Maintaining this distance during sphere rotation proved impractical.

3. Hardware Availability: The sensors available were outdated and some were physically damaged. Newer optical sensors were difficult to source and costly.

Due to these limitations, the optical mouse sensor approach was deemed unreliable for continuous and accurate sphere orientation tracking.

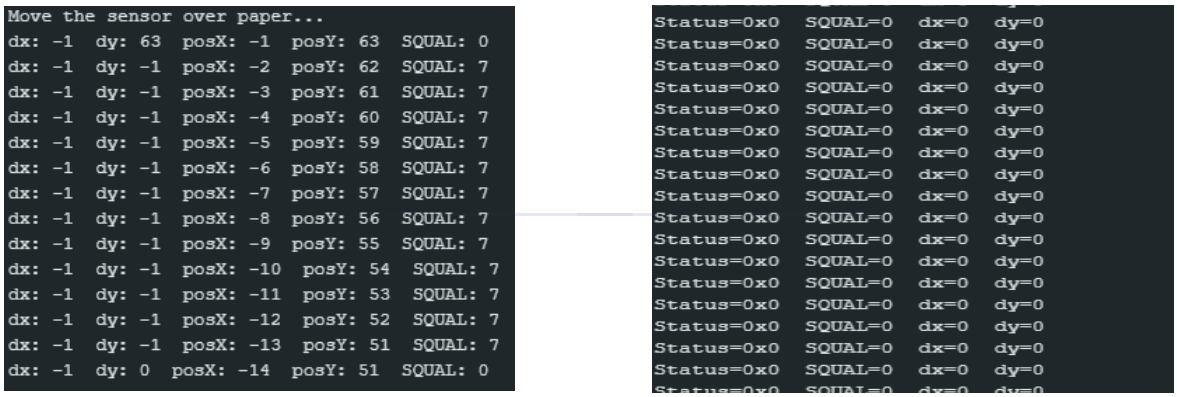


Figure 6. Example output values from the mouse sensor.

3.5 Transition to ArUco Marker-Based Computer Vision

After concluding that optical mouse sensors were not suitable for reliable sphere tracking, an alternative sensing approach was required. The decision was made to implement a fiducial marker-based tracking system using ArUco markers and a global shutter camera.

3.5.1 Why ArUco Markers?

ArUco markers provide several advantages over color-based or feature-based tracking:

- Unique Identification: Each marker has a unique binary pattern, allowing unambiguous identification
- Robust Detection: Built-in error correction and detection algorithms in OpenCV
- Precise Corner Localization: Corner refinement enables accurate pose estimation
- Dictionary-Based: Pre-defined dictionaries prevent false positive detections

3.5.2 Hardware Configuration

The camera-based sensing system consists of:

Component	Specification	Purpose
Raspberry Pi 5	8GB RAM	Processing unit
Arducam OV9281	1MP, Global Shutter, Monochrome, NoIR, M12 lens	Marker detection

Third Floor Plate	Custom mount	Camera mounting + light shielding
22-to-22 Pin Cable	CSI ribbon	Camera interface

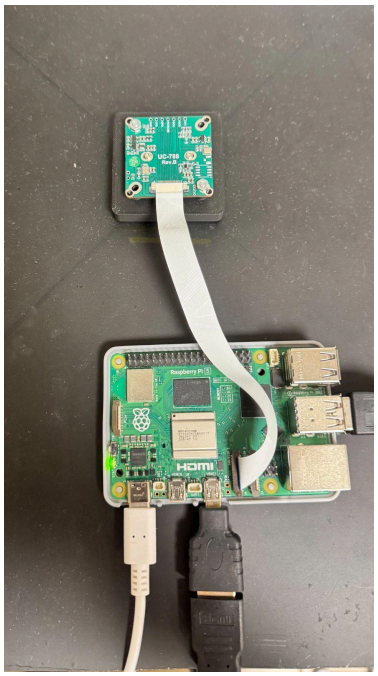


Figure 7. Sensor system hardware.

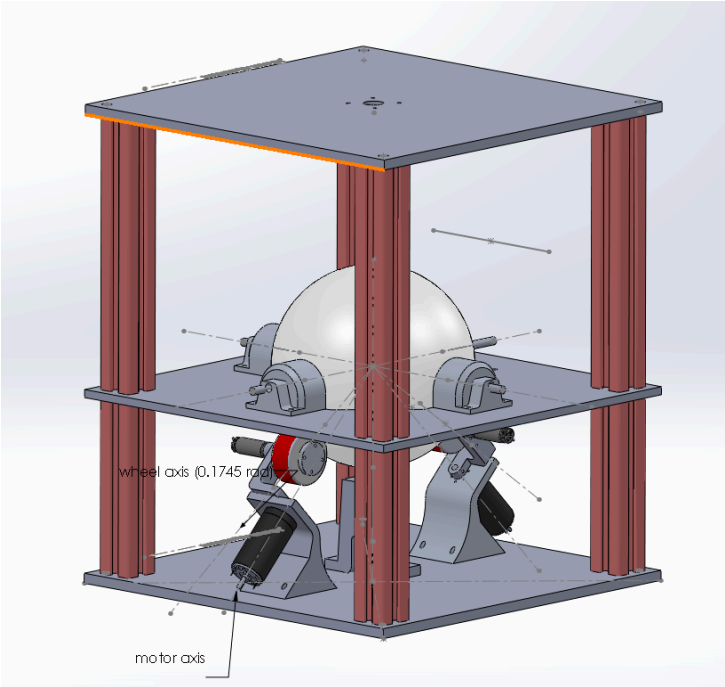


Figure 8. Mechanical system with the 3rd plate added.

Why Global Shutter?

Rolling shutter cameras capture images row-by-row, causing temporal distortion during motion. When the sphere rotates, a rolling shutter would distort marker geometry:

Rolling Shutter vs. Global Shutter Effect on Propeller:

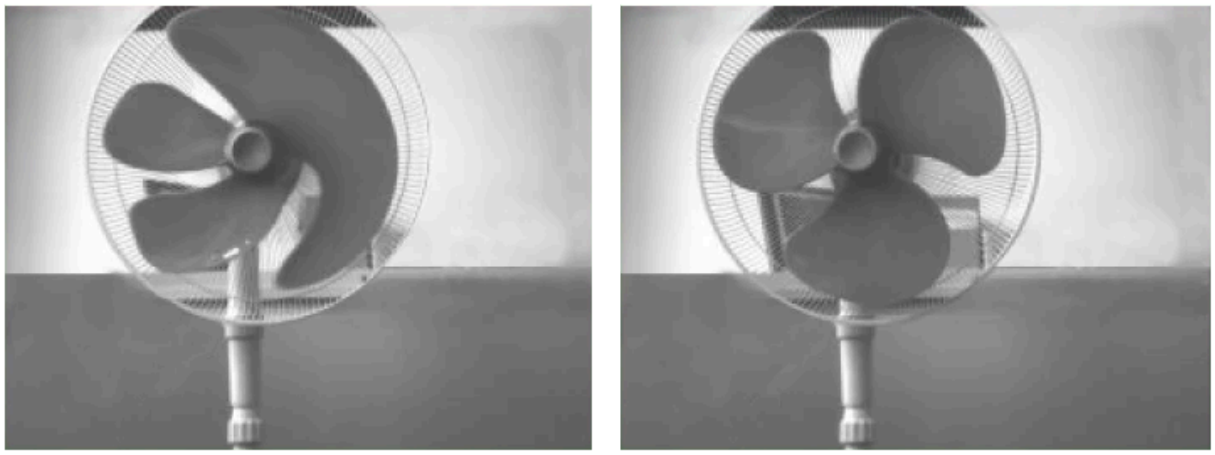


Figure 7. Comparison betwenn global and non-global shutter camera images.

The OV9281's global shutter captures all pixels simultaneously (exposure time ~1000µs), ensuring marker corners remain geometrically accurate during rotation.



Figure 8. OV9281 camera.

3.5.3 Marker Distribution Strategy

We use the ArUco dictionary DICT_5X5_50 with all 50 markers (IDs 0-49) distributed on the sphere surface. The distribution ensures that at least 3 markers are visible to the camera at any sphere orientation.

Why minimum 3 markers?

- 1 marker: Provides only 1 bearing vector (2 DOF) — insufficient
- 2 markers: 4 DOF constraints, but rotation ambiguity exists around the axis connecting them
- 3+ markers: Fully constrains all 3 rotational DOF

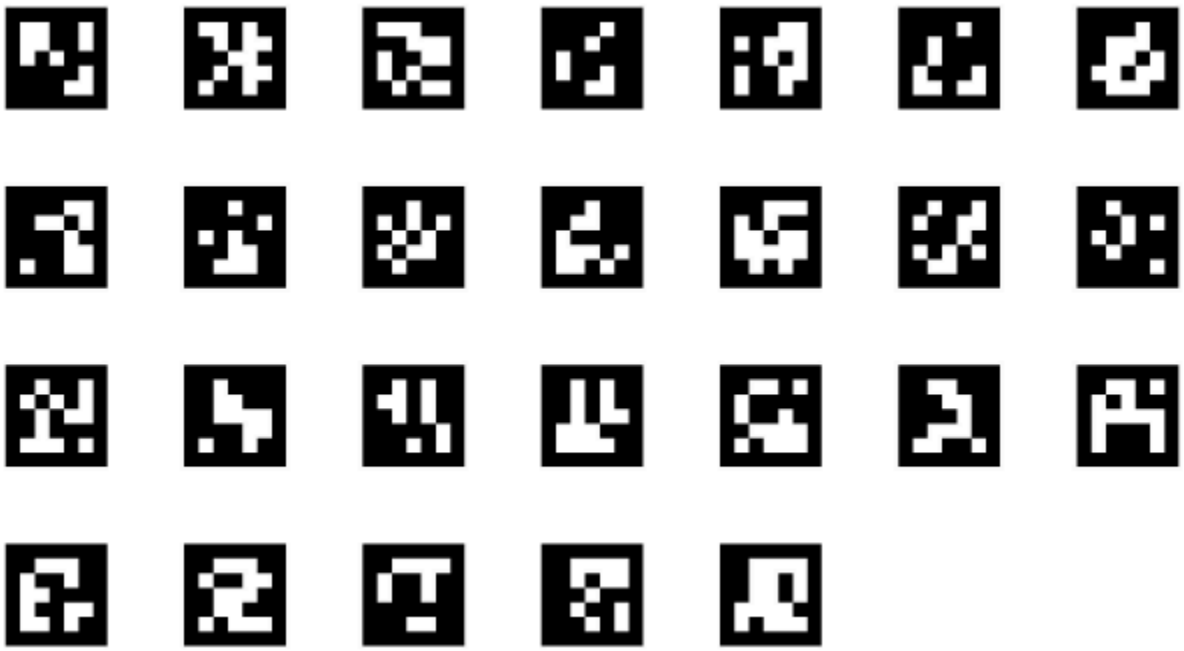


Figure 9. ArUCO markers 24-49.

3.6 Camera Calibration

Before tracking can begin, the camera's intrinsic parameters must be calibrated using a chessboard pattern.

3.6.1 Pinhole Camera Model

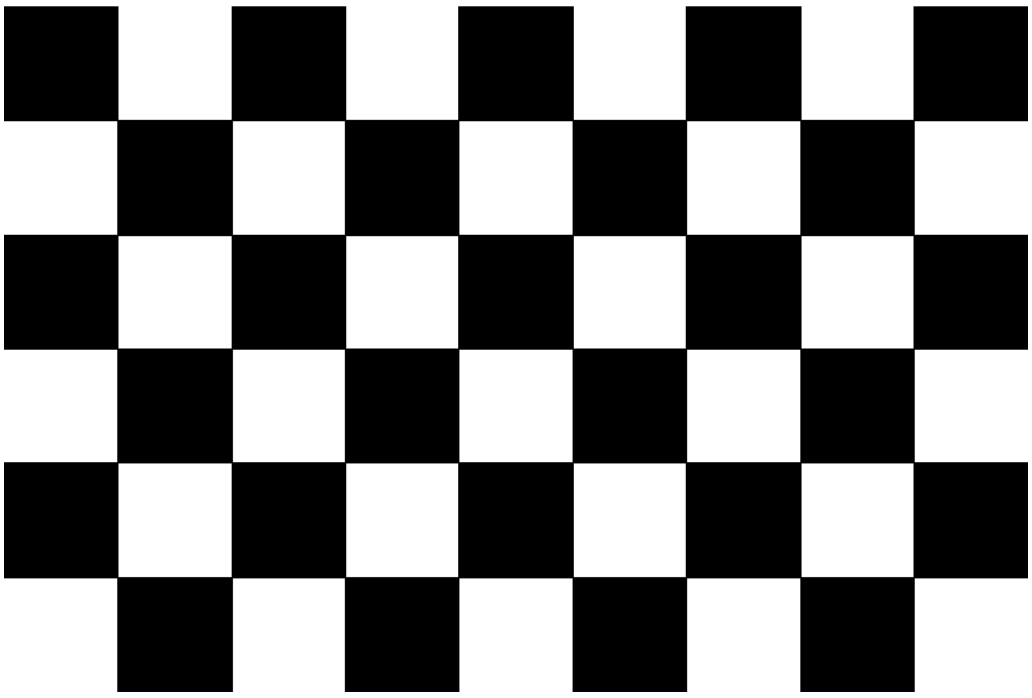
The camera is modeled using the pinhole projection model. A 3D point P = [X, Y, Z]^T in camera coordinates projects to pixel coordinates [u, v]^T via:

where the **intrinsic matrix K** is:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Parameters:

- f_x, f_y: Focal lengths in pixel units (typically f_x = f_y for square pixels)
- (c_x, c_y): Principal point where optical axis intersects image plane



3.6.2 Lens Distortion Model

Real lenses introduce distortion, especially wide-angle lenses. The distortion is modeled using radial and tangential coefficients:

$$\text{dist} = [k_1,\ k_2,\ p_1,\ p_2,\ k_3]$$

Figure 13. Distortion calculation.

For a point with normalized coordinates (x_u, y_u), define:

$$r^2 = x_u^2 + y_u^2$$

Figure 14. Defined radius with normalized coordinates.

Radial distortion:

$$x_r = x_u \left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6\right)$$

$$y_r = y_u \left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6\right)$$

Figure 15. Radial distortion.

Tangential distortion:

$$x_d = x_r + 2p_1 x_u y_u + p_2 \left(r^2 + 2x_u^2\right)$$

$$y_d = y_r + p_1 \left(r^2 + 2y_u^2\right) + 2p_2 x_u y_u$$

Figure 16. Tangential distortion.

```
{
  "imageSize": {
    "width": 1280,
    "height": 800
  },
  "patternSize": {
    "inner_x": 8,
    "inner_y": 5
  },
  "squareSize_m": 0.0192,
```

```
  "K": [
    [
      956.0794648729449,
      0.0,
      627.5171360698678
    ],
    [
      0.0,
      953.0302796940398,
      387.0332650222431
    ],
    [
      0.0,
      0.0,
      1.0
    ]
  ],
```

```
  "dist": [
    0.03176982165198837,
    0.09361272025258102,
    0.0022542525059579825,
    4.1394859885032124e-06,
    -0.099743022480821
  ],
  "reprojection_error": 1.0687320975557004,
  "used_images": 21,
  "total_images": 29
}
```


Figure 17. K_dist parameters

3.6.3 Calibration Procedure

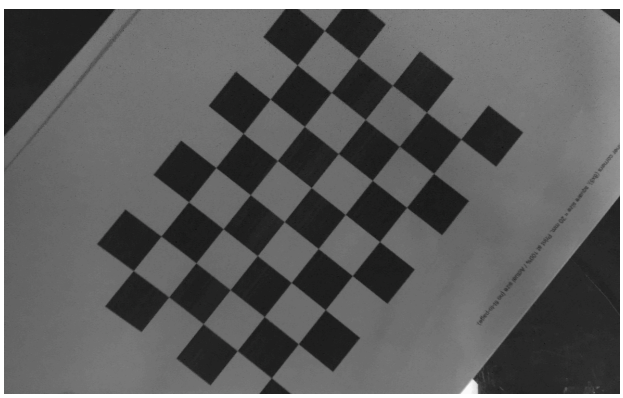
The calibration uses Zhang's method with an 8×5 chessboard pattern:

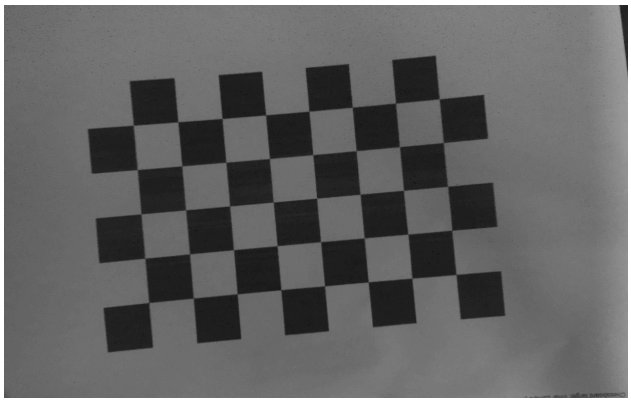
1. Capture 20-30 images of the chessboard at various orientations
2. Detect inner corners using cv2.findChessboardCorners()
3. Refine corners to sub-pixel accuracy using cv2.cornerSubPix()
4. Solve for K and dist using cv2.calibrateCamera()

The optimization minimizes **reprojection error**:

$$E = \sum_{i} \sum_{j} \left\| p_{ij} - \hat{p}\left(K,\ \text{dist},\ R_i,\ t_i,\ P_j\right) \right\|^2$$

Figure 18. Reprojection error minimization.





Achieved Results:

- Reprojection error: < 0.5 pixels
- Output file: camera_calib_chessboard.json

3.7 Sphere Geometry and Marker Position Acquisition

3.7.1 3D Scanning of the Sphere

To achieve high tracking accuracy, the precise 3D positions of all markers were obtained by 3D scanning the physical sphere using a structured light scanner. The scan data was processed in CloudCompare software.

Sphere Primitive Fitting:

A sphere primitive was fitted to the point cloud, yielding:

- **Sphere center**: C_scan = [-1.24242, -39.6743, -24.412] mm
- **Sphere radius**: R_scan = 100.352486 mm

3.7.2 Marker Corner Extraction

For each of the 50 ArUco markers, the four corner positions were manually picked from the 3D scan. The marker center is computed as:

$$p_{\text{center}} = \frac{1}{4} \sum_{i=1}^{4} p_{\text{corner},i}$$

Figure 20. Marker center computation.

3.7.3 Unit Vector Computation

Each marker's position is converted to a **unit direction vector** in the sphere's coordinate frame:

$$\mathbf{v}_{\text{sphere}} = \frac{p_{\text{center}} - C_{\text{scan}}}{\left\| p_{\text{center}} - C_{\text{scan}} \right\|}$$

Figure 21. Unit direction vector.

This normalization is valid because all markers lie on the sphere surface, so:

$$\left\| p_{\text{center}} - C_{\text{scan}} \right\| \approx R_{\text{scan}}$$

Figure 22. Normalization of the mentioned vector conversion.

Advantage over Latitude/Longitude:

Previous approaches used spherical coordinates (latitude φ, longitude λ), with using mobile spirit level:

$$\mathbf{v} = \begin{bmatrix} \cos(\varphi)\cos(\lambda) \\ \cos(\varphi)\sin(\lambda) \\ \sin(\varphi) \end{bmatrix}$$

This requires trigonometric calculations every frame. Our approach:

- Pre-computes unit vectors **once** at startup
- Eliminates sin/cos calls leads to 30× computational speedup

- Higher accuracy from direct measurement

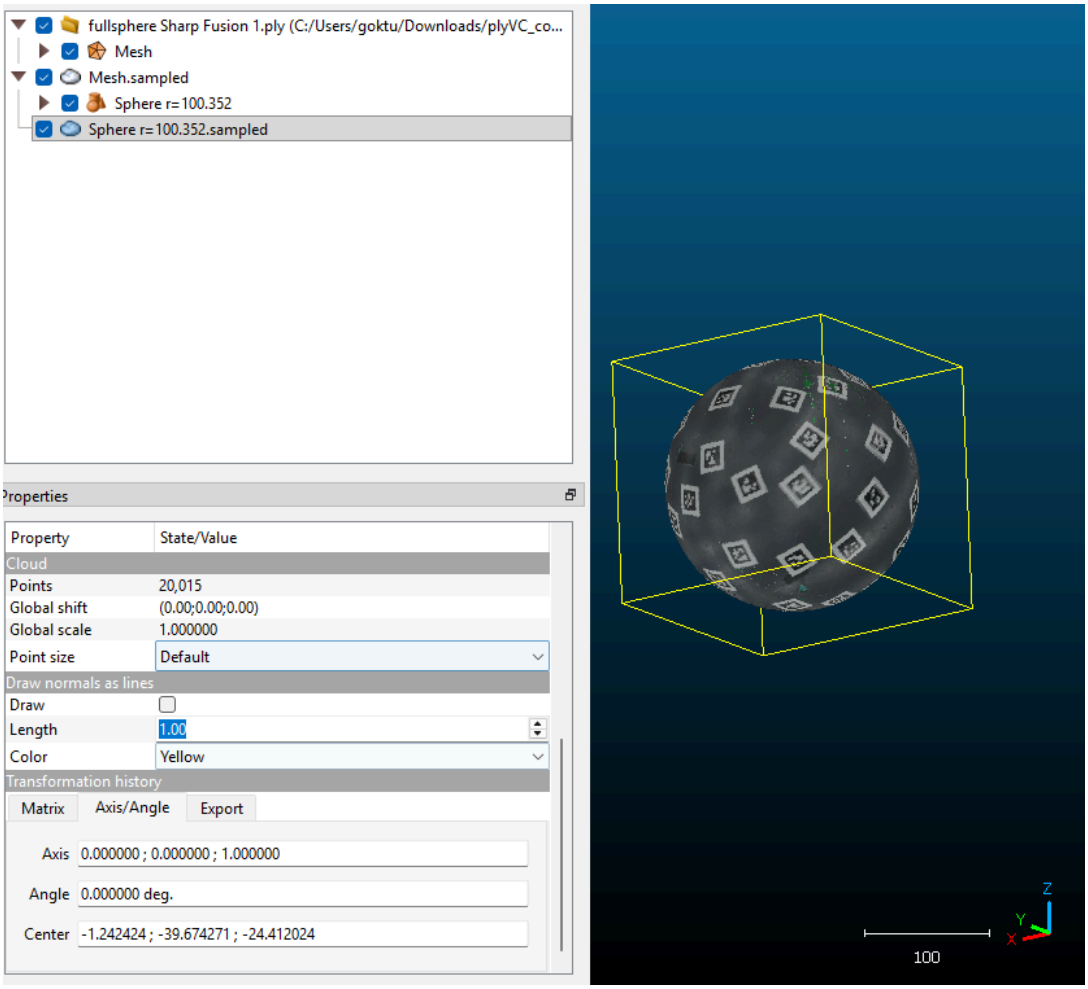


Figure 23. 3D Model of the sphere.

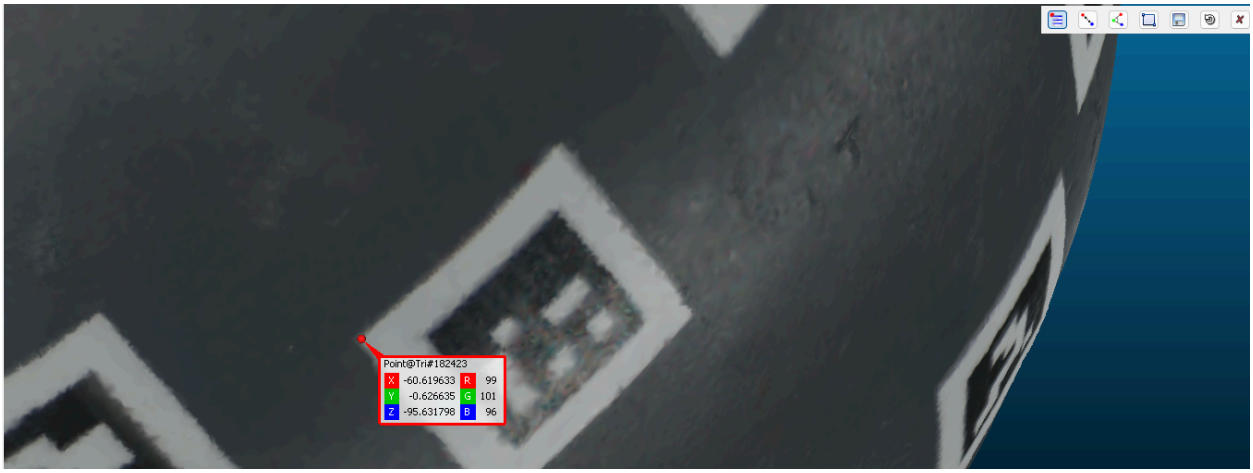


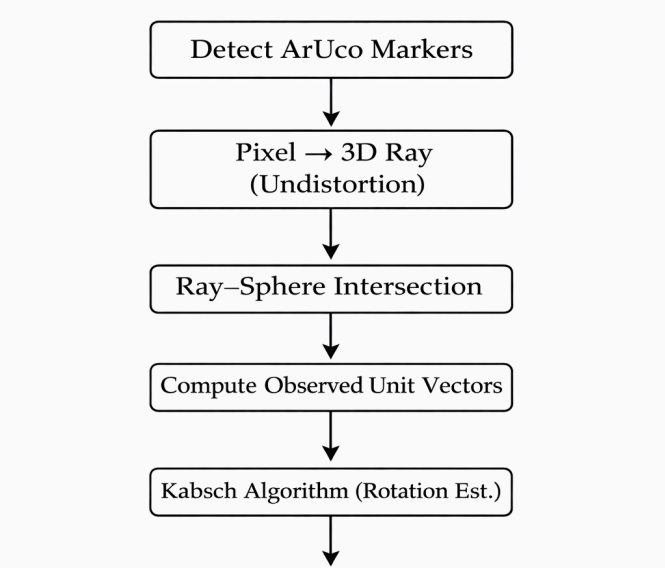
Figure 24. Specifying the corners of ArUCO markers.

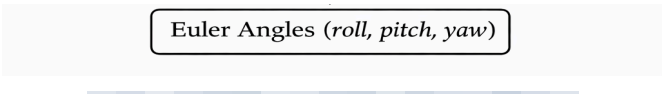
3.8 Core Tracking Algorithm: Mathematical Foundations

The tracking algorithm solves a rigid-body attitude estimation problem using bearing-only (angle/unit) observations constrained to a known sphere.

3.8.1 Algorithm Pipeline Overview

For each frame:





3.8.2 Step 1: Pixel to 3D Ray Conversion

Given a detected marker center at pixel coordinates (u, v), we compute the corresponding 3D ray direction in camera coordinates.

Substep 1a: Undistort the pixel

The pixel is first corrected for lens distortion using OpenCV's cv2.undistortPoints():

$$\begin{bmatrix} x_{\text{norm}} \\ y_{\text{norm}} \end{bmatrix} = \text{undistort}(u,\ v,\ \mathbf{K},\ \text{dist})$$

This yields **normalized image coordinates** — the point on the z=1 plane.

Substep 1b: Construct ray direction

$$\mathbf{r} = \begin{bmatrix} x_{\text{norm}} \\ y_{\text{norm}} \\ 1.0 \end{bmatrix}$$

Substep 1c: Normalize to unit vector

$$\hat{\mathbf{r}} = \frac{\mathbf{r}}{\|\mathbf{r}\|}$$

3.8.3 Step 2: Sphere Center in Camera Coordinates

Since the camera and sphere are fixed relative to each other, the sphere center C_cam in camera coordinates is estimated once using similar triangles.

Depth Estimation:

$$Z = f_x \times \left(\frac{R_{\text{actual}}}{R_{\text{pixel}}} \right)$$

where:

- R_actual = 0.10035 m (from 3D scan)
- R_pixel = sphere radius in pixels (from calibration)
- f_x = focal length from camera calibration (to find how many pixels covers 1 rad in frame)

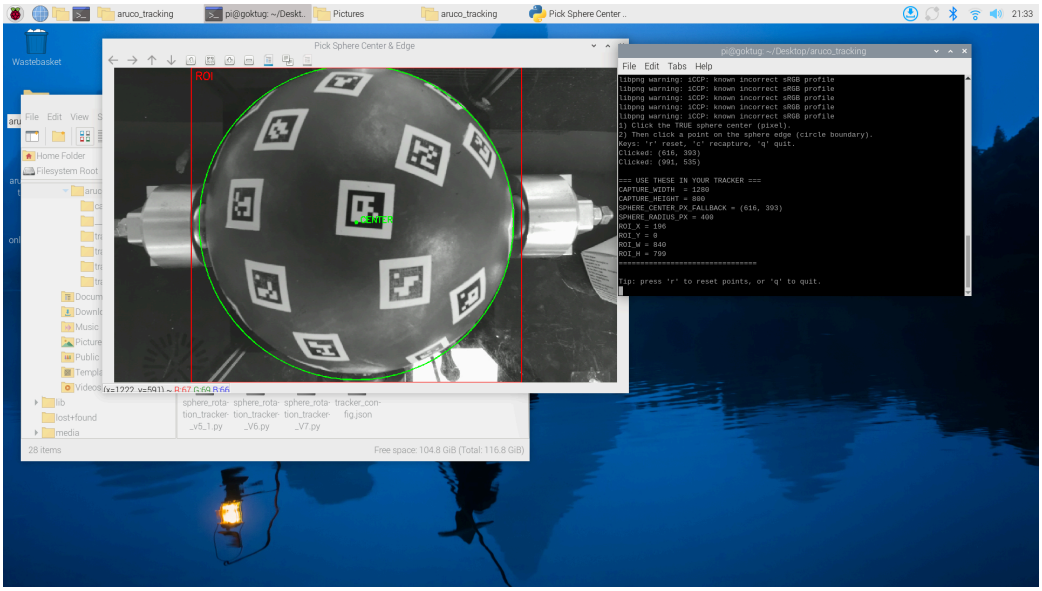


Figure 26. Selecting the center point of the sphere with the pick_center_roi.py.

Center Position:

The sphere center lies along the ray through the center pixel:

$$\mathbf{C}_{\text{cam}} = \hat{\mathbf{r}}_{\text{center}} \times \left(\frac{Z}{\hat{\mathbf{r}}_{\text{center},z}} \right)$$

3.8.4 Step 3: Ray-Sphere Intersection

For each marker, we find where the camera ray intersects the sphere surface.

Problem Setup:

Given:

- Ray origin: O = (0, 0, 0) (camera at origin)
- Ray direction: $\hat{r}$ (unit vector)
- Sphere center: C
- Sphere radius: R

Find: Point P on sphere surface where $P = t \times \hat{r}$ for some $t > 0$.

Mathematical Derivation:

The sphere surface satisfies:

$$\|P - C\|^2 = R^2$$

Substituting $P = t \times \hat{r}$:

$$\|t\times\hat{r} - C\|^2 = R^2$$

Expanding the squared norm:

$$(t\times\hat{r} - C) \cdot (t\times\hat{r} - C) = R^2$$

$$t^2(\hat{r}\cdot\hat{r}) - 2t(\hat{r}\cdot C) + (C\cdot C) = R^2$$

Since $\hat{r}$ is a unit vector, $\hat{r}\cdot\hat{r} = 1$:

$$t^2 - 2t(\hat{r}\cdot C) + \|C\|^2 - R^2 = 0$$

This is a **quadratic equation** $at^2 + bt + c = 0$ with:

- $a = 1$
- $b = -2(\hat{r}\cdot C)$
- $c = \|C\|^2 - R^2$

Solution using Quadratic Formula:

$$\Delta = b^2 - 4ac = b^2 - 4c$$

Case 1: $\Delta < 0 \rightarrow$ Ray misses sphere (no real intersection)

Case 2: $\Delta \geq 0 \rightarrow$ Two intersection points:

$$t_{1,2} = (-b \pm \sqrt{\Delta}) / 2$$

We select the **smaller positive** value (front intersection):

$$t = \min\{t_i : t_i > 0\}$$

The intersection point is:

$$P_cam = t \times \hat{r}$$

Implementation:

```
def ray_sphere_intersection(ray, C, r):
    b = -2.0 * float(np.dot(ray, C))
    c = float(np.dot(C, C) - r*r)
    disc = b*b - 4.0*c

    if disc <= 0:
        return None  # Ray misses sphere

    s = sqrt(disc)
    t1 = (-b - s) / 2.0
    t2 = (-b + s) / 2.0

    # Select smallest positive t
    t = None
    for cand in (t1, t2):
        if cand > 0 and (t is None or cand < t):
            t = cand

    if t is None:
```

return None

return t * ray # 3D point on sphere

3.8.5 Step 4: Observed Unit Vector Computation

The intersection point P_cam lies on the sphere surface. We convert this to a unit direction vector from the sphere center:

v_cam = (P_cam - C_cam) / R

Why is this a unit vector?

By definition, P_cam is on the sphere surface, so:

‖P_cam - C_cam‖ = R

Therefore:

$$v_{\text{cam}}(i) = \frac{P_{\text{cam}}(i) - C_{\text{cam}}}{R} \qquad \|P_{\text{cam}}(i) - C_{\text{cam}}\| = R$$

$$\|v_{\text{cam}}(i)\| = 1$$

3.8.6 Step 5: The Correspondence Problem

For each visible marker i, we now have a correspondence pair:

Known (from sphere 3D scan)	Observed (from camera)
$v_{\text{sph}}(i)$	$v_{\text{cam}}(i)$

The relationship between them is a rotation:

$$v_{\text{cam}}(i) = R\, v_{\text{sph}}(i)$$

where R is the 3×3 rotation matrix from sphere frame to camera frame.

3.8.7 Step 6: Kabsch Algorithm for Rotation Estimation

With N ≥ 3 marker correspondences, we solve for the optimal rotation matrix R.

Problem Formulation (Wahba's Problem, 1965):

Find R that minimizes:

$$J(R) = \sum_{i=1}^{N} \left\| R\, v_{\text{sph}}(i) - v_{\text{cam}}(i) \right\|^2$$

Why not just solve directly?

With noise and measurement errors, no single R perfectly satisfies all equations. The Kabsch algorithm finds the least-squares optimal rotation.

Kabsch Algorithm Steps:

Step 6a: Construct data matrices

Stack the unit vectors into matrices:

$$V_{\text{sph}} = \begin{bmatrix} v_{\text{sph}}(1)^T \\ v_{\text{sph}}(2)^T \\ \vdots \\ v_{\text{sph}}(N)^T \end{bmatrix} \in \mathbb{R}^{N\times 3} \qquad V_{\text{cam}} = \begin{bmatrix} v_{\text{cam}}(1)^T \\ v_{\text{cam}}(2)^T \\ \vdots \\ v_{\text{cam}}(N)^T \end{bmatrix} \in \mathbb{R}^{N\times 3}$$

Step 6b: Compute cross-covariance matrix

$$H = V_{\text{sph}}^T\, V_{\text{cam}}$$

This 3×3 matrix captures the correlation structure between the two vector sets:

$$H = \sum_{i=1}^{N} v_{\text{sph}}(i)\, v_{\text{cam}}(i)^T$$

Step 6c: Singular Value Decomposition (SVD)

Decompose H:

$$H = U\, \Sigma\, V^T$$

where:

- U is a 3×3 orthogonal matrix
- Σ is a 3×3 diagonal matrix of singular values
- V is a 3×3 orthogonal matrix

SVD decomposes any matrix into: **Rotate** → **Scale** → **Rotate**

Step 6d: Compute rotation matrix

$$R = V\, U^T$$

This formula extracts the pure rotational component, discarding the scaling (which represents noise/errors).

Step 6e: Handle reflection case

If det(R) = -1, the result is a reflection, not a rotation. Fix by negating the last column of V:

If det(R) < 0:

$$R = V \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & -1 \end{bmatrix} U^T$$

Implementation:

```
def kabsch_rotation(V_from, V_to):
    """
    Compute optimal rotation using Kabsch algorithm.
    """
    # Cross-covariance matrix
    H = V_from.T @ V_to

    # SVD decomposition
    U, S, Vt = np.linalg.svd(H)

    # Rotation matrix
    R = Vt.T @ U.T

    # Ensure proper rotation (not reflection)
    if np.linalg.det(R) < 0:
        Vt[-1, :] *= -1
        R = Vt.T @ U.T

    return R
```

3.8.8 Step 7: Rotation Matrix to Euler Angles

The rotation matrix R must be converted to human-interpretable Euler angles (roll, pitch, yaw).

ZYX Convention (Aerospace Standard):

The rotation is decomposed as sequential rotations:

$$R = R_z(\psi)\ R_y(\theta)\ R_x(\phi)$$

where:

- φ = **roll** (rotation about X-axis)
- θ = **pitch** (rotation about Y-axis)
- ψ = **yaw** (rotation about Z-axis)

Why order matters:

Rotations are **non-commutative**: $R_x \times R_y \neq R_y \times R_x$

Different conventions (XYZ, ZXZ) yield different angles for the same rotation matrix. We use ZYX for compatibility with QUARC/Simulink.

Extraction Formulas:

For rotation matrix:

$$R = \begin{bmatrix} r_{00} & r_{01} & r_{02} \\ r_{10} & r_{11} & r_{12} \\ r_{20} & r_{21} & r_{22} \end{bmatrix}$$

First, compute:

$$s_y = \sqrt{r_{00}^2 + r_{10}^2}$$

Non-singular case ($s_y > \varepsilon$, typically $\varepsilon = 10^{-6}$):

$\theta = \text{atan2}(-r_{20}, s_y)$ (pitch)
$\psi = \text{atan2}(r_{10}, r_{00})$ (yaw)
$\varphi = \text{atan2}(r_{21}, r_{22})$ (roll)

Singular case (gimbal lock) ($s_y \leq \varepsilon$):

When $\theta = \pm 90°$, roll and yaw become coupled:

$\psi = \text{atan2}(-r_{01}, r_{11})$
$\varphi = 0$

Implementation:

```
def rot_to_euler_zyx(R):
    """Convert rotation matrix to Euler angles (ZYX convention)."""
    sy = sqrt(R[0,0]**2 + R[1,0]**2)
    singular = sy < 1e-6
```

```
if not singular:
    roll  = atan2(R[2,1], R[2,2])
    pitch = atan2(-R[2,0], sy)
    yaw   = atan2(R[1,0], R[0,0])
else:
    roll  = 0.0
    pitch = atan2(-R[2,0], sy)
    yaw   = atan2(-R[0,1], R[1,1])

return np.degrees([roll, pitch, yaw])
```

3.9 Robustness and Filtering Techniques

Raw tracking output contains noise and discontinuities. Two filtering stages improve robustness.

3.9.1 Spatial Gating: Radius Limit Filter

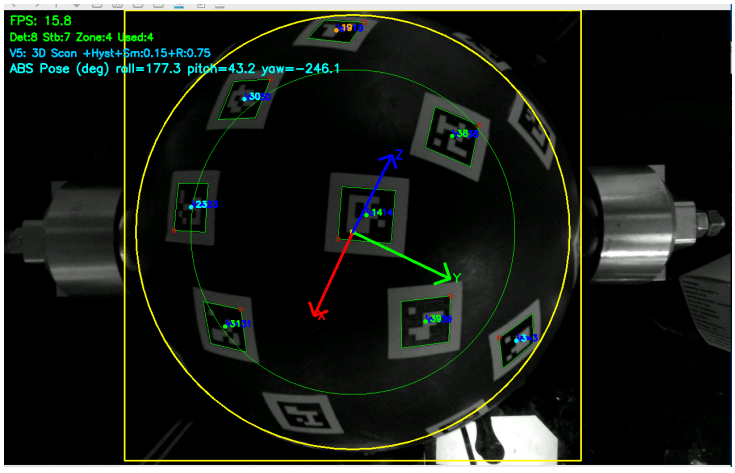
Problem: Markers near the sphere's silhouette (edge as seen from camera) suffer from:

- Perspective foreshortening
- Surface curvature distorting marker appearance
- Geometric ill-conditioning (small pixel errors → large angular errors)

Solution: Only use markers within 75% of the sphere radius from center:

$$d = \sqrt{[(u - c_x)^2 + (v - c_y)^2]}$$

Marker is used **only if**: $d \leq 0.75 \times R_{pixel}$



3.9.2 Temporal Filtering: Marker Hysteresis

Problem: Markers at detection threshold flicker on/off between frames, causing pose jumps.

Solution: State-machine-based hysteresis with two thresholds:

HYST_FRAMES_TO_ADD = 3: Marker must be detected 3 consecutive frames to become "active"

HYST_FRAMES_TO_REMOVE = 5: Marker must be missing 5 consecutive frames to become "inactive"

3.10 System Integration and Communication

3.10.1 Software Architecture

The system consists of three main components:

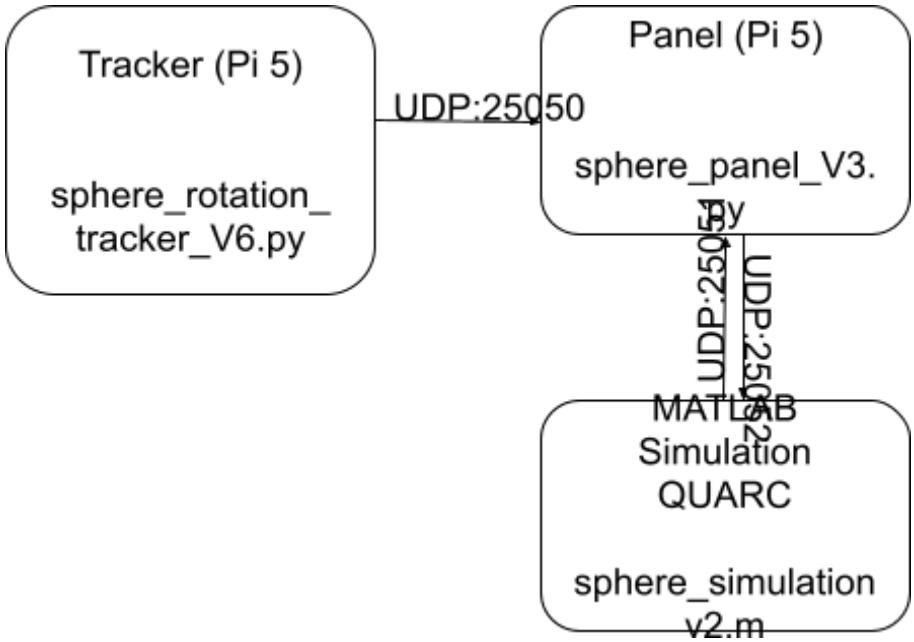


Figure 28. Architecture of software.

3.10.2 UDP Communication Protocol

Tracker → Panel/MATLAB (Port 25050):

JSON packet at 30 Hz:

{
 "roll": 12.34,
 "pitch": -5.67,
 "yaw": 89.01,
 "detected": 8,
 "used": 6,
 "fps": 29.8
}

3.10.3 Drift Computation

The Panel computes drift as the Euclidean distance between camera and encoder angles:

$$\text{drift} = \sqrt{(\phi_{\text{enc}} - \phi_{\text{cam}})^2 + (\theta_{\text{enc}} - \theta_{\text{cam}})^2 + (\psi_{\text{enc}} - \psi_{\text{cam}})^2}$$

Threshold-based status:

- **OK**: drift < 6°
- **WARNING**: 6° ≤ drift < 10°
- **ALARM**: drift ≥ 10°

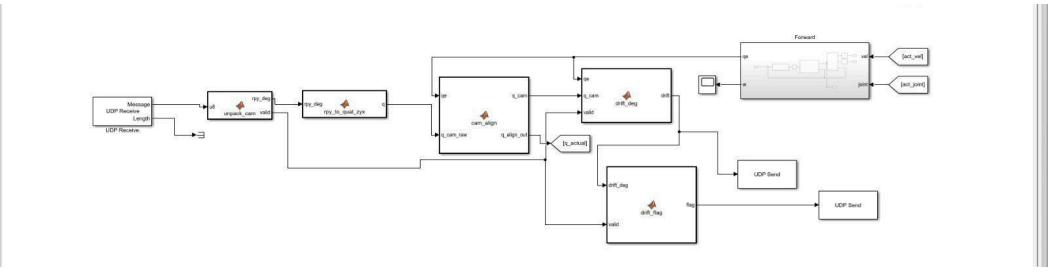
3.10.4 MATLAB Drift Injection Simulation

To validate the drift detection system without requiring QUARC hardware, a MATLAB simulation was developed.

Follow Camera Mode:

1. Receive camera pose from tracker (port 25050)
2. Add configurable drift rate (deg/s)
3. Add optional noise
4. Send as "encoder pose" to panel (port 25051)

This simulates the exact symptom of the drift problem: encoder estimate diverges from camera measurement over time.



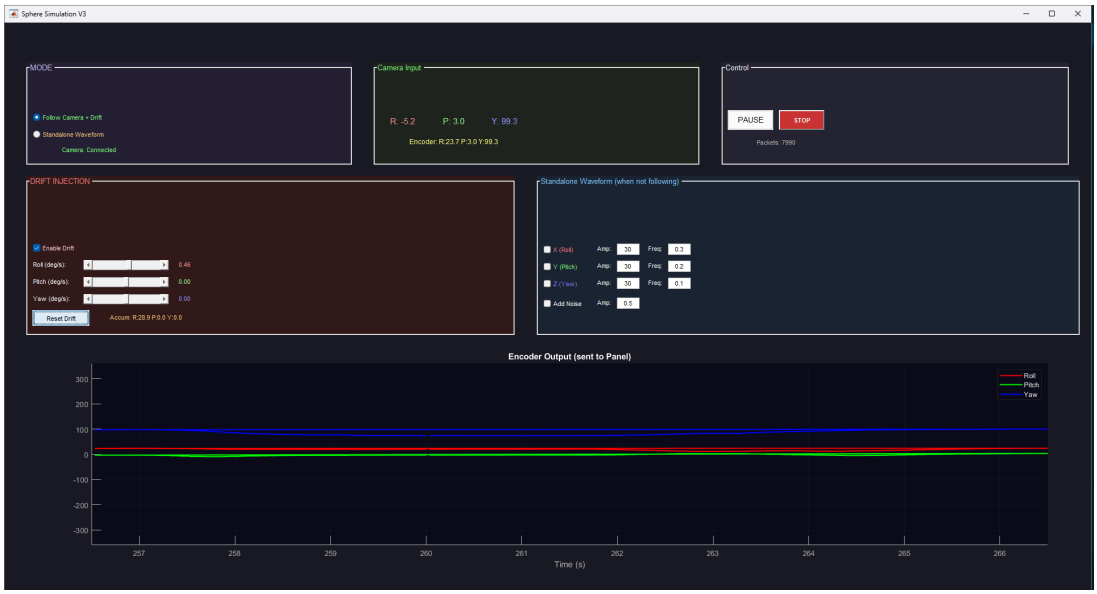


Figure 30. sphere_simulationv2.m copying the QUARCs communication.

3.11 Summary of Mathematical Pipeline

Step	Input	Operation	Output
1	Pixel (u,v)	Undistort + normalize	Ray direction $\hat{r}$
2	$\hat{r}$, sphere geometry	Quadratic equation	Intersection point P_cam
3	P_cam, C_cam	Normalization	Observed unit vector v_cam
4	N correspondences	Kabsch (SVD)	Rotation matrix R
5	R	Euler extraction	Roll, pitch, yaw

6	Raw angles	EMA smoothing	Filtered angles

This mathematical framework transforms 2D pixel observations into 3D orientation estimates with sub-degree accuracy, enabling precise drift detection between encoder estimates and actual sphere motion.

4. RESULTS & DISCUSSION

This section presents a detailed analysis of the experimental results obtained using the final camera-based tracking system. The performance of the system is evaluated both visually and quantitatively, and the results are discussed with respect to the original project objectives.

4.1 Achievement of Initial Objectives

The main objective of this project was to measure the real orientation of the spherical manipulator independently from the motor encoders and to observe the drift caused by mechanical effects such as wheel slip and imperfect contact. This objective was successfully achieved in the final system. The implemented vision-based tracking method is able to estimate the sphere's orientation directly from camera observations, without relying on encoder integration. As a result, the system can provide an absolute orientation reference and prevent unbounded error accumulation.

In addition to basic orientation estimation, the system can now compare encoder-based and camera-based orientation values in real time. This allows drift to be clearly observed and quantified during motion, which directly addresses the main limitation of previous versions of the project.

4.2 Rotation Tracking Results

The experimental results show that the system can track rotations around all three axes of the sphere. Rotations in roll, pitch, and yaw were successfully detected and monitored. During the experiments, the sphere was rotated not only around a single axis, but also with combined motions involving multiple axes at the same time.

The obtained orientation plots demonstrate smooth and continuous angle changes. No sudden jumps or unstable behavior were observed. The direction and magnitude of the rotations match the physical motion of the sphere, which confirms that the algorithm estimates the global 3D orientation correctly. These results indicate that the tracking system is robust under realistic motion conditions.

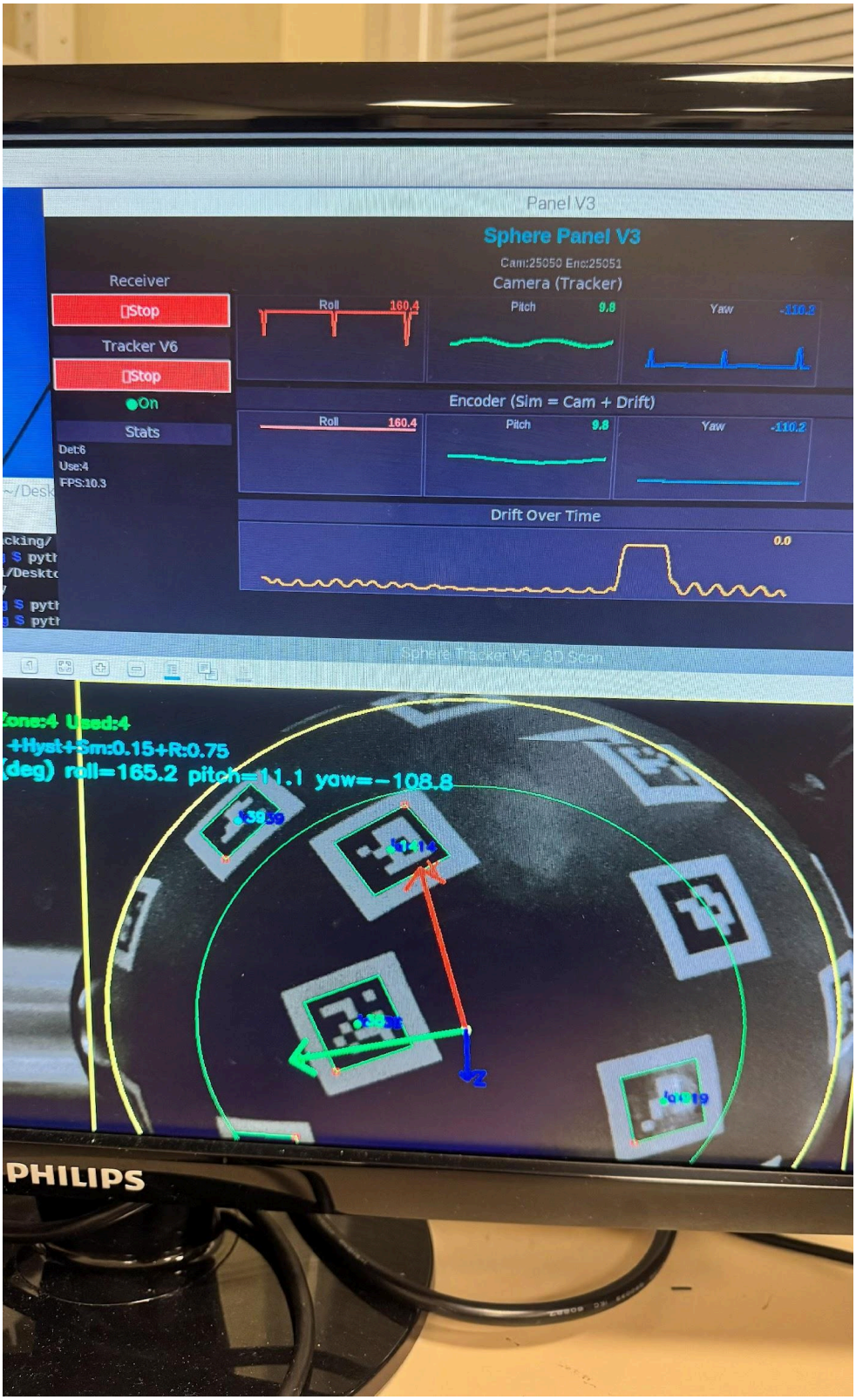


Figure 31. X rotation (Roll) with drift shown on Raspberry Pi 5 interface.

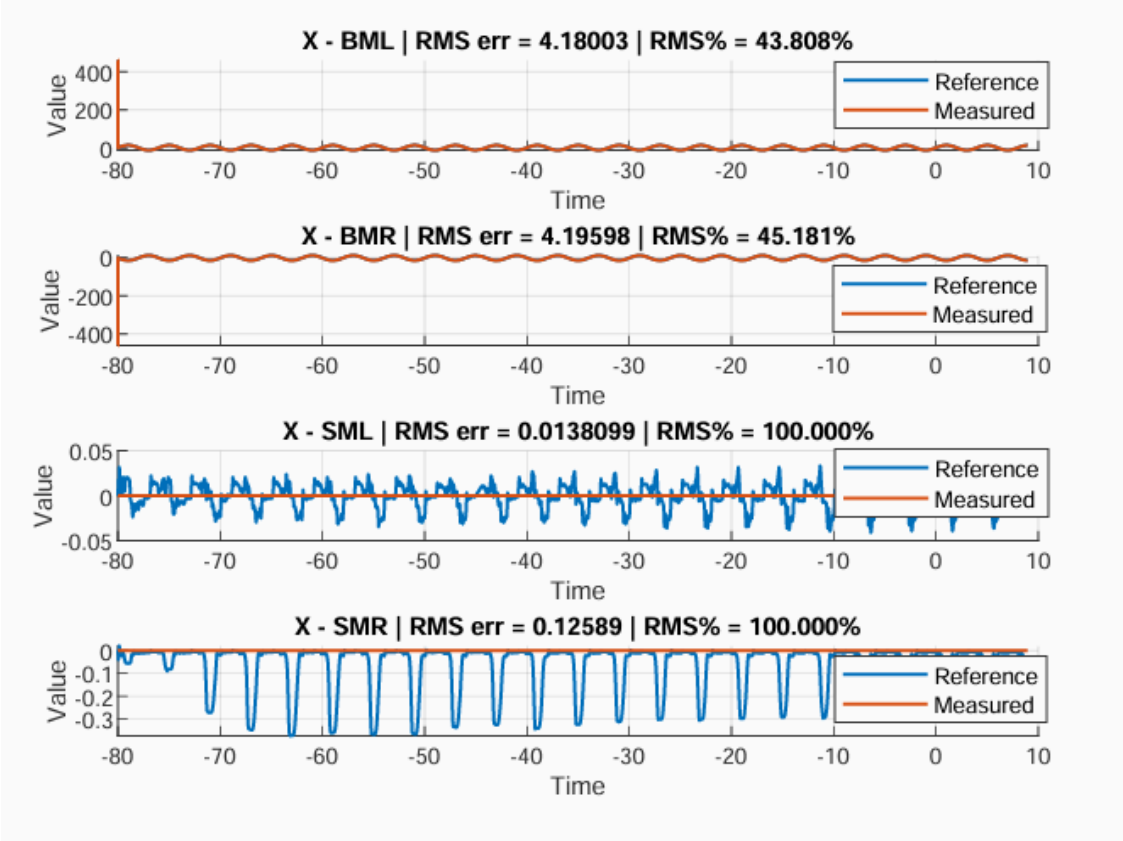


Figure 32. X rotation encoder data with RMS error value.

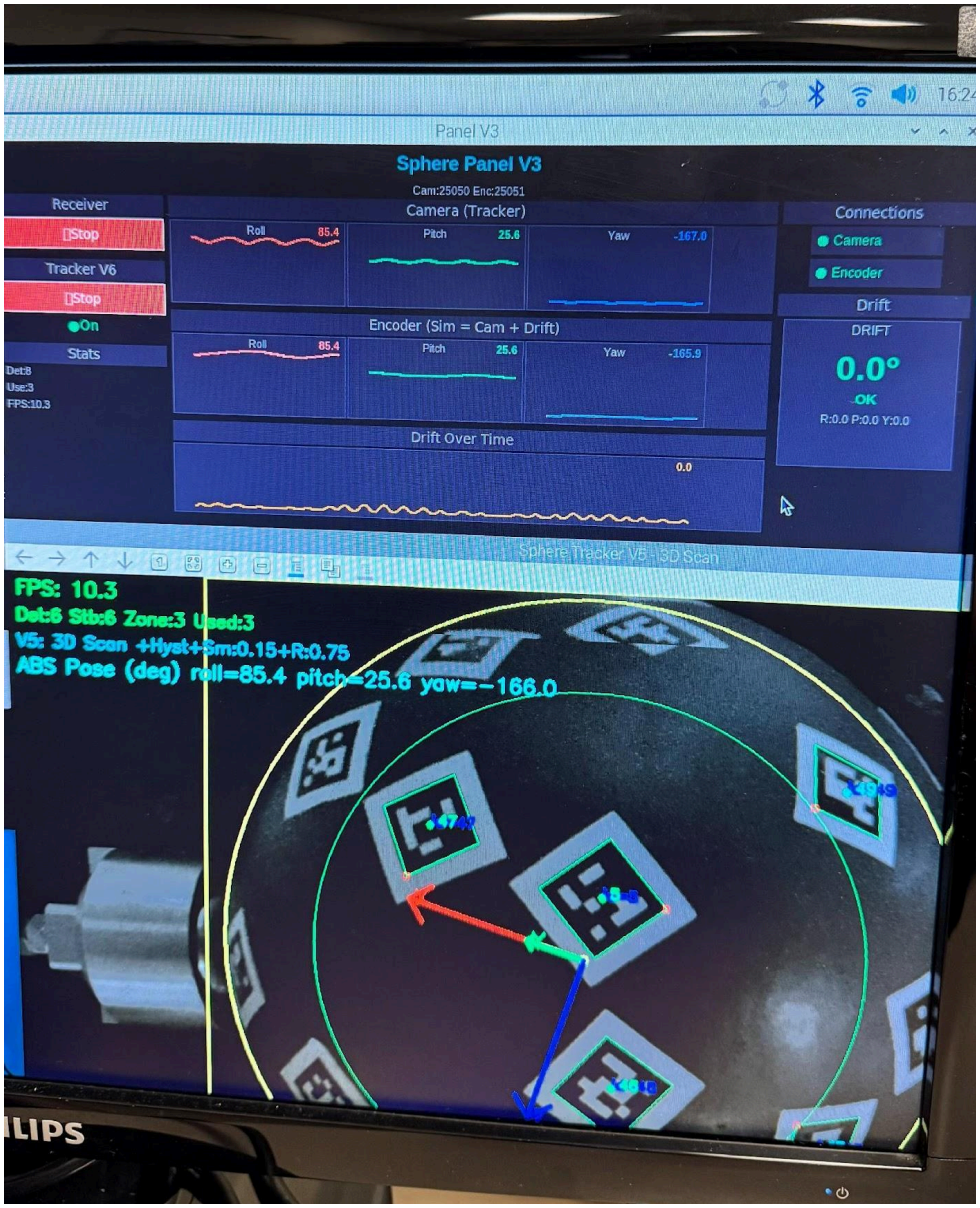


Figure 33. Y rotation (Pitch) with drift shown on Raspberry Pi 5 interface.

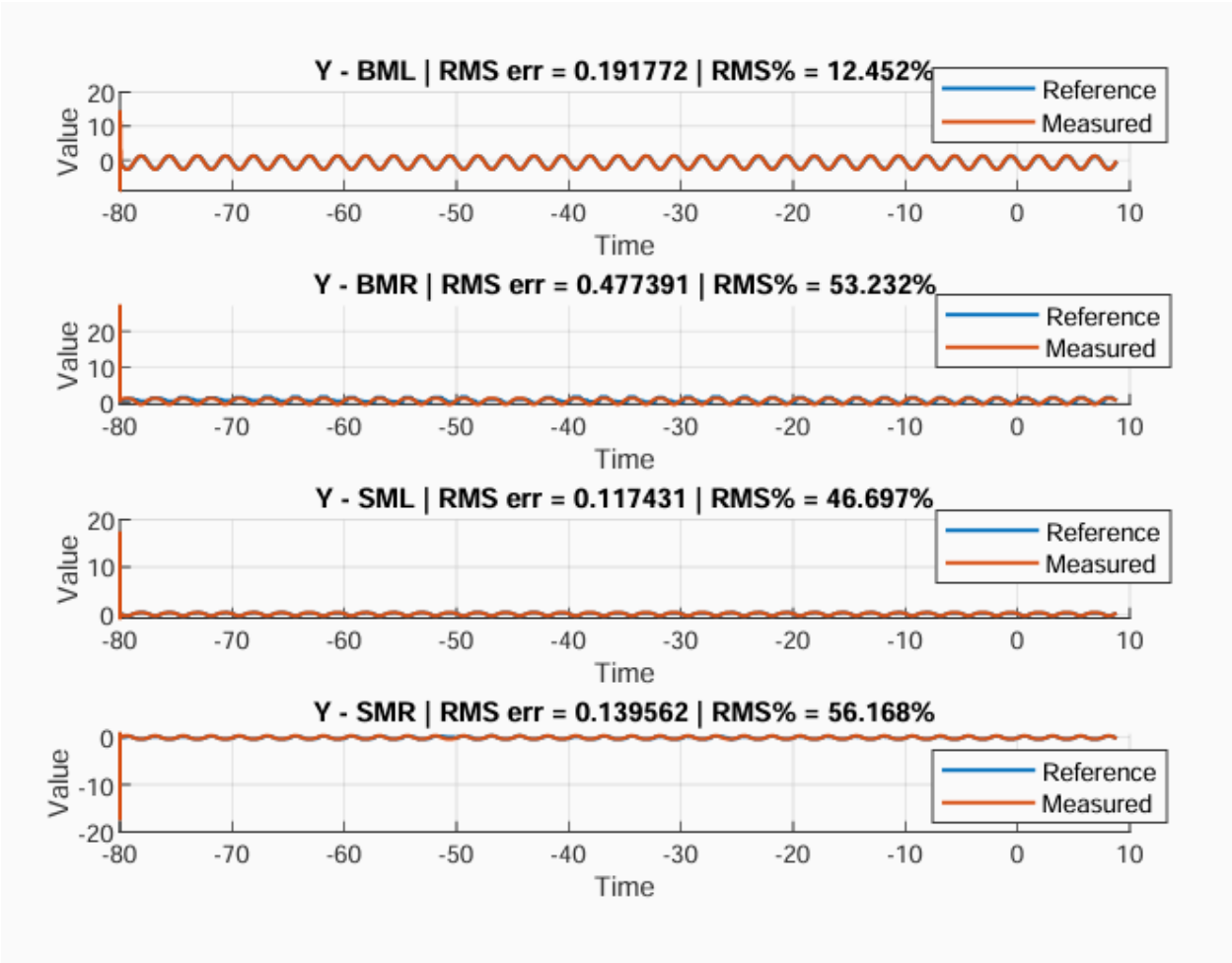
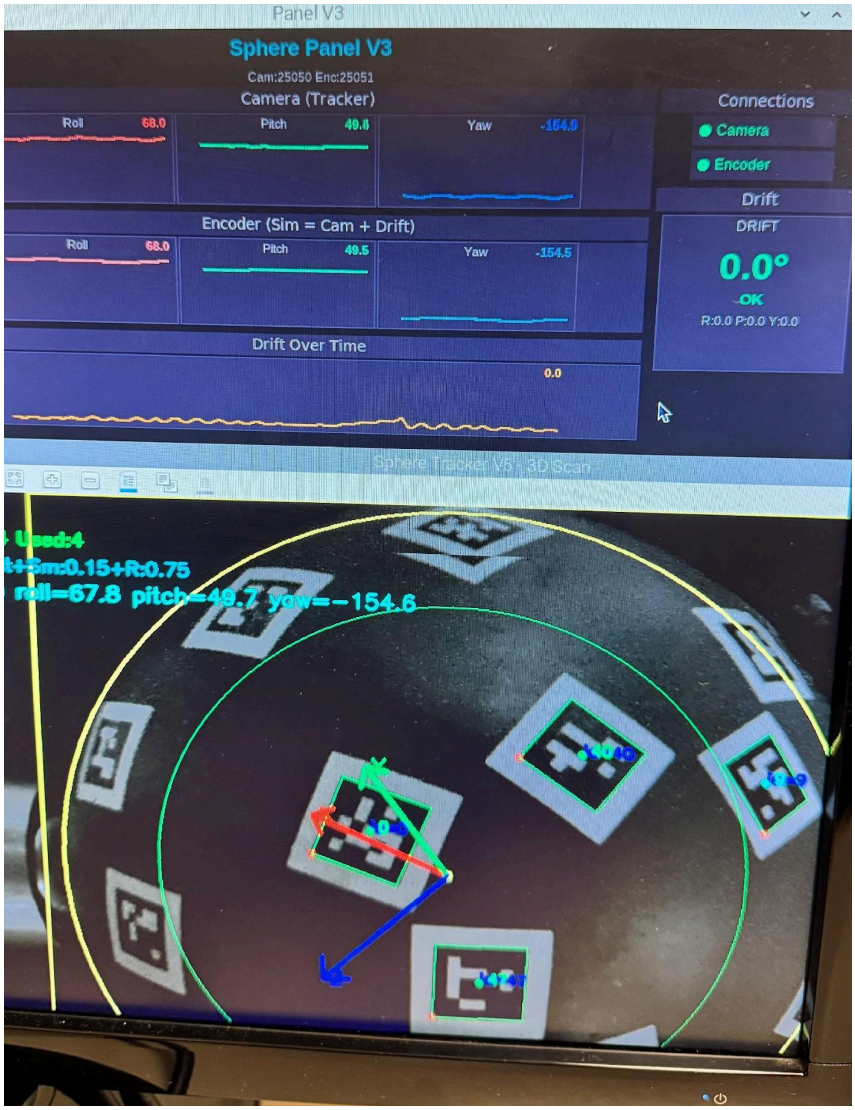


Figure 34. Y rotation encoder data.


Panel V3
Sphere Panel V3
Cam:25050 Enc:25051
Camera (Tracker)
Roll 68.0
Pitch 49.8
Yaw -154.0
Encoder (Sim = Cam + Drift)
Roll 68.0
Pitch 49.5
Yaw -154.5
Drift Over Time
0.0
Connections
Camera
Encoder
Drift
DRIFT
0.0°
OK
R:0.0 P:0.0 Y:0.0
Used:4
roll=67.8 pitch=49.7 yaw=-154.6

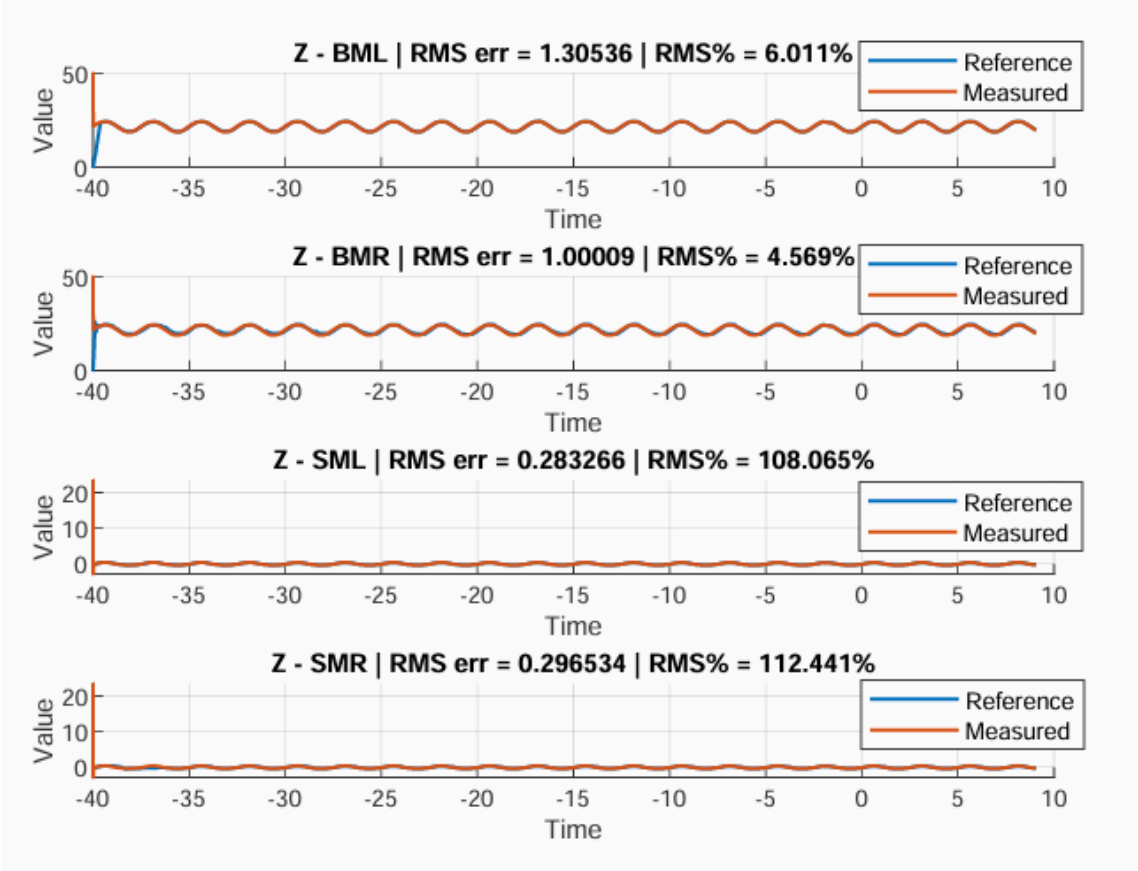


Figure 36. Z rotation encoder data.

Figure: Yaw (z) rotation with drift



Figure 37. Complex rotation with drift shown on Raspberry Pi 5 interface.

4.3 Drift Detection and Monitoring

To test the drift detection capability, artificial drift was added to the encoder-based orientation values. This simulated real-world problems such as wheel slip and accumulated encoder error. While the encoder-based orientation gradually deviated from the true motion, the vision-based orientation remained stable.

The difference between the two orientation estimates was calculated as a drift value. The results show that the system successfully captures how drift grows over time. The measured drift follows the injected disturbance closely, which verifies that the system can reliably detect and monitor encoder drift during operation.

4.4 Comparison With the Initial Sensor-Based Approach

Compared to the initial expectations of the project, the final system provides more functionality than originally planned. The original goal focused mainly on sensing the sphere's orientation, while the final system also includes real-time drift monitoring and validation of encoder reliability. The ability to

track full 3D motion under combined rotations represents a significant improvement over earlier approaches that were limited to simpler motion patterns.

4.5 Project Completion Assessment

Based on the achieved results, the project can be considered successfully completed. All major objectives were met, including full 3D orientation tracking, drift detection, and stable operation during continuous motion. Although closed-loop drift correction was not implemented, the system provides all the required information for such control strategies to be added in future work..

4.6 Contribution and Engineering Insight

This project makes a meaningful contribution by demonstrating that encoder drift in spherical manipulators can be directly measured instead of being assumed or ignored. By combining a vision-based, bearing-only sensing approach with known sphere geometry, the system provides a reliable absolute reference for orientation. This makes the platform more suitable for educational use, where accurate and understandable behavior is important.

Overall, the results show that the developed system successfully improves the reliability and observability of the spherical manipulator and bridges the gap between theoretical models and real-world behavior.

4.7 Limitations and Future Work

Despite the positive results, several limitations remain:

- Sensitivity to lighting conditions and reflections,
- Raspberry Pi 5 not being a powerful enough computer to handle all processes related to the sensor system and leading to low fps imaging.
- Our QUARC license does not contain reach to specific QUARC blocks that would enable us to process the angle error to a usable form for us to implement the orientation data we receive from the optical camera sensor.
- The BML operates with a significantly lower torque than BMR which is thought to be an effect after many years of use. It leads to the motor not being able to recover itself after it rotates to certain angles in both directions and begins to oscillate uncontrollably due to the weight of cables and the wheel overpowering the torque it is able to produce.

Overall, the camera based sensor setup proved it can precisely monitor the rotation of the sphere and if the required software support is delivered, it can be used to compensate for the drift of

the system. Further improvements can be done around the components to enhance the performance further, such as the Raspberry Pi 5 with a development board that can withstand heavier workload, and our Arducam camera with a higher performance model.

The aged components are candidates for updates as well if near-perfect performance of the system is seeked.

5. IMPACT

Our parallel spherical manipulator project is a novel research on the robotics field as an education oriented haptic device that uses a spherical end effector. The end product is presumed to pave the way for deeper understanding over parallel spherical kinematics, rotations. As it is planned to be used in Sabancı University robotics course lectures with the previously mentioned motives, there will be a big user pool who will experience and experiment with the system.

Using Optical cameras and pattern recognition to measure the rotations of a non-translating sphere is another topic that has literature gaps which we were unable to find much information about.

Our project does not concern any Freedom to Use (FTU) issues.

6. ETHICAL ISSUES

There are no current ethical issues related to the project as the project is a novel effort that has not been worked on or researched on before..

7. PROJECT MANAGEMENT

When we first began working on the project our first objective was to comprehend what had been done before starting with the kinematic and dynamic analysis of the system. Our plans progression included working on the hardware setup for maintenance and to see whether any complications existed that would hinder our work. Comprehending and upgrading the previously made joint control algorithm was another task. With these tasks done, we would have started working on the sensor system that would push the project another step towards being finished and succeeding as an educational purpose haptic device.

Our plans had to be altered due to complications we encountered throughout the year. Hardware setup and optimization of it cost us more time than presumed. Comprehension over previous

years' work has also taken most time out of any other task that we scheduled. Due to our group only consisting of 2 people and us not being sufficiently competent over control engineering when we first began our work on this project made it harder to comprehend and getting used to the control architecture previously made. Also as further upgrades on it were necessary, this step was crucial and was addressed deeply.

Due to these changes in the time schedule of the project, the Graphical User Interface that the previous group worked on was never addressed and the main focus was on implementing the sensor system only.

The necessary sensor setup was built successfully and is able to track the rotations of the sphere; however, as the required QUARC blocks cannot be used with the license provided, the data from received from the optical sensor cannot be implemented into the control algorithm.

However, a simulation was built to represent the meaning of the sensor mechanism built when drift is monitored.

8. CONCLUSION AND FUTURE WORK

After the completion of the project, the educational use of it will begin in Sabancı University to complement the lecture content about Spherical robots and rotations. The project being built as a haptic device with real time control will enable students to increase their understanding over the content with visualization and self-controlled experiments.

9. APPENDIX

A. flow_code

```
// ADNS-2610 + Arduino Nano

// VCC -> 5V, GND -> GND, SCK -> D13, SDIO -> D11


const int PIN_SCK  = 13;

const int PIN_SDIO = 11;
```

```
long posX = 0;

long posY = 0;


void setSDIOOutput() { pinMode(PIN_SDIO, OUTPUT); }

void setSDIOInput()  { pinMode(PIN_SDIO, INPUT);  }


// Clock out one byte, MSB first

void writeByte(uint8_t b) {

  setSDIOOutput();

  for (int i = 7; i >= 0; i--) {

          digitalWrite(PIN_SCK, LOW);

          digitalWrite(PIN_SDIO, (b & (1 << i)) ? HIGH : LOW);

          delayMicroseconds(1);

          digitalWrite(PIN_SCK, HIGH);

          delayMicroseconds(1);

  }

}
```

```
// Read one byte, MSB first
uint8_t readByte() {
  uint8_t val = 0;
  setSDIOInput();
  for (int i = 7; i >= 0; i--) {
        digitalWrite(PIN_SCK, LOW);
        delayMicroseconds(1);
        digitalWrite(PIN_SCK, HIGH);
        if (digitalRead(PIN_SDIO)) val |= (1 << i);
        delayMicroseconds(1);
  }
  return val;
}

uint8_t adnsRead(uint8_t addr) {
  // first byte: 0 + 7-bit address (read)
  writeByte(addr & 0x7F);
  // give sensor time to prepare data (datasheet: >= 100 us)
  delayMicroseconds(150);
```

```
  return readByte();
}

void adnsWrite(uint8_t addr, uint8_t data) {
  // first byte: 1 + 7-bit address (write)
  writeByte(0x80 | (addr & 0x7F));
  delayMicroseconds(150);
  writeByte(data);
  delayMicroseconds(150);
}

void setup() {
  pinMode(PIN_SCK, OUTPUT);
  digitalWrite(PIN_SCK, LOW);
  pinMode(PIN_SDIO, OUTPUT);
  digitalWrite(PIN_SDIO, HIGH);

  Serial.begin(115200);
  delay(50);
```

```
  // Reset + keep sensor awake (Configuration register 0x00, bits: reset & forced awake)
  adnsWrite(0x00, 0x81);
  delay(50);

  uint8_t status = adnsRead(0x01); // Status
  Serial.print("Status: 0x");
  Serial.println(status, HEX);

  Serial.println("Move the sensor over paper...");
}

void loop() {
  // Read deltas (reading clears them)
  int8_t dy = (int8_t)adnsRead(0x02); // Delta_Y
  int8_t dx = (int8_t)adnsRead(0x03); // Delta_X

  // Update position
  posX += dx;
```

```
posY += dy;



// Surface quality

uint8_t squal = adnsRead(0x04);       // SQUAL



// Only print if something interesting happened

if (dx != 0 || dy != 0) {

        Serial.print("dx: ");

        Serial.print(dx);

        Serial.print("  dy: ");

        Serial.print(dy);

        Serial.print("  posX: ");

        Serial.print(posX);

        Serial.print("  posY: ");

        Serial.print(posY);

        Serial.print("  SQUAL: ");

        Serial.println(squal);

}
```

```
  delay(5);
}
```

B. sphere_rotation_tracker_V6

```
#!/usr/bin/env python3
"""
Sphere Rotation Tracker V5
--------------------------
Major Update: Uses direct 3D coordinates from 3D scan data
- No more lat/lon conversion - direct unit vectors from scanned marker positions
- Sphere center from fitted sphere primitive in CloudCompare
- ~30x computational improvement by eliminating trigonometric calculations
- All 50 ArUco markers with precise coordinates

Coordinate System:
- Sphere center C from fitted sphere: (-1.24242, -39.6743, -24.412) mm
- Sphere radius R from fitted sphere: 100.352486 mm
- Marker positions projected onto sphere surface as unit vectors
"""

import os
import cv2
```

```
import numpy as np
import subprocess
import time
import json
import csv
import sys
import socket
import struct
from math import sqrt, atan2

# ============================================================================
# 1) CONFIGURATION - Load from file or use defaults
# ============================================================================
CONFIG_FILE = "tracker_config.json"

DEFAULT_CONFIG = {
    "shutter": 1000,
    "gain": 8,
    "center_x": 620,
    "center_y": 391,
    "radius_px": 394,
    "roi_x": 206,
    "roi_y": 0,
    "roi_w": 828,
```

```
    "roi_h": 799,
    "framerate": 30,
    "capture_width": 1280,
    "capture_height": 800
}

def load_config():
    """Load configuration from JSON file, with defaults for missing values."""
    config = DEFAULT_CONFIG.copy()
    if os.path.exists(CONFIG_FILE):
        try:
            with open(CONFIG_FILE, 'r') as f:
                loaded = json.load(f)
                config.update(loaded)
            print(f"[CONFIG] Loaded from {CONFIG_FILE}", file=sys.stderr)
        except Exception as e:
            print(f"[CONFIG] Error loading {CONFIG_FILE}: {e}, using defaults", file=sys.stderr)
    else:
        print(f"[CONFIG] {CONFIG_FILE} not found, using defaults", file=sys.stderr)
    return config

# Load config at startup
CFG = load_config()
```

```
# Apply config values
FRAMERATE = CFG["framerate"]
SHUTTER = CFG["shutter"]
GAIN = CFG["gain"]

CAPTURE_WIDTH = CFG["capture_width"]
CAPTURE_HEIGHT = CFG["capture_height"]

SPHERE_CENTER_PX_FALLBACK = (CFG["center_x"], CFG["center_y"])
SPHERE_RADIUS_PX = CFG["radius_px"]
ROI_X = CFG["roi_x"]
ROI_Y = CFG["roi_y"]
ROI_W = CFG["roi_w"]
ROI_H = CFG["roi_h"]

# ============================================================================
# 2) ArUco Config
# ============================================================================
ARUCO_DICT_TYPE = cv2.aruco.DICT_5X5_50
SPHERE_RADIUS_M = 0.10035  # 100.35mm from 3D scan
MARKER_STICKER_SIZE_M = 0.022  # 22mm white square

# ============================================================================
# 3) DETECTION TUNING
```

```
# ============================================================================

USE_CLAHE = False

CLAHE_CLIP_LIMIT = 2.0

CLAHE_TILE_SIZE = (8, 8)

USE_CORNER_REFINEMENT = True


# ============================================================================

# 3.1) MARKER HYSTERESIS - Prevents jitter from markers appearing/disappearing

# ============================================================================

# A marker must be seen for HYST_FRAMES_TO_ADD consecutive frames to be "active"

# A marker must be missing for HYST_FRAMES_TO_REMOVE consecutive frames to be "inactive"

HYSTERESIS_ENABLED = True

HYST_FRAMES_TO_ADD = 3      # Frames needed to start using a marker

HYST_FRAMES_TO_REMOVE = 5   # Frames needed to stop using a marker


# ============================================================================

# 3.2) TEMPORAL SMOOTHING - Low-pass filter for pose output

# ============================================================================

SMOOTHING_ENABLED = True

SMOOTHING_ALPHA = 0.15      # 0.0 = full smoothing (slow), 1.0 = no smoothing (raw)

                    # Lower = smoother but more latency

                    # Recommended: 0.2-0.4 for stable display


# ============================================================================
```

```
# 3.3) DETECTION RADIUS LIMIT - Only use markers within trusted zone

# ============================================================================

# Markers near the edge of the sphere have more perspective distortion

# Only use markers within this fraction of the sphere radius from center

RADIUS_LIMIT_ENABLED = True

RADIUS_LIMIT_FRACTION = 0.75   # 0.5 = only use markers within 50% of radius from center

                    # Very strict - only center markers used

                    # Adjust: 0.4=very strict, 0.6=moderate, 0.75=loose


# ============================================================================

# 3.4) UDP OUTPUT - Stream pose data to Panel and MATLAB

# ============================================================================

UDP_OUTPUT_ENABLED = True


# Target 1: Panel (localhost)

UDP_TARGET_IP = "127.0.0.1"

UDP_TARGET_PORT = 25050


# Target 2: MATLAB PC (for simulation to receive camera data)

UDP_TARGET_IP_2 = "172.20.10.2"   # MATLAB PC IP - değiştir!

UDP_TARGET_PORT_2 = 25050        # Same port, different machine


UDP_SEND_RATE_HZ = 30
```

```
# ============================================================================

# 4) Files

# ============================================================================

CALIB_FILE = "camera_calib_chessboard.json"

OUTPUT_DIR = "tracking_output_v5"

CSV_FILE = os.path.join(OUTPUT_DIR, "pose_log_v5.csv")



# ============================================================================

# 5) 3D Scan Data - Sphere Center and Radius from CloudCompare

# ============================================================================

# Sphere center from fitted sphere primitive (mm)

SPHERE_CENTER_SCAN = np.array([-1.24242, -39.6743, -24.412], dtype=np.float64)

# Sphere radius from fitted sphere (mm)

SPHERE_RADIUS_SCAN = 100.352486



# ============================================================================

# 6) Raw 3D Marker Corner Coordinates from 3D Scan (mm)

# Each marker has 4 corner points picked in CloudCompare

# Format: marker_id: [(x1,y1,z1), (x2,y2,z2), (x3,y3,z3), (x4,y4,z4)]

# ============================================================================

MARKER_CORNERS_RAW = {

  0: [

    (77.691864, -98.904434, -41.893127),

    (78.084869, -87.438446, -62.275013),
```

(90.323181, -69.648048, -52.295643),

(89.507896, -81.291435, -31.635748),

],

1: [

(-53.843472, -125.505020, -30.373138),

(-36.594147, -133.790176, -16.954741),

(-23.141241, -137.207245, -35.800652),

(-40.609936, -128.781204, -49.221657),

],

2: [

(-69.341026, -20.109308, 46.108372),

(-78.018631, -40.985165, 39.487007),

(-62.136765, -51.983761, 53.898026),

(-53.907570, -30.849049, 60.105438),

],

3: [

(44.978031, 25.525410, 36.779743),

(47.107296, 38.156536, 16.854923),

(66.403030, 25.849874, 11.329046),

(63.906036, 12.970741, 30.766319),

],

4: [

(-11.904025, -47.577103, -123.936935),

(-10.277898, -23.065020, -122.947090),

(13.518112, -25.244635, -122.551453),

(11.758102, -48.893082, -123.789352),

],

5: [

(51.356316, -124.510803, -33.028069),

(33.741013, -133.672668, -19.521282),

(20.210514, -136.452362, -38.643585),

(37.848076, -127.440445, -52.371716),

],

6: [

(-46.187252, 46.365425, -50.700962),

(-54.974495, 33.083702, -68.345802),

(-72.287498, 24.905989, -54.439369),

(-63.709610, 38.185028, -36.384304),

],

7: [

(19.791466, 58.540306, -32.681164),

(19.157808, 57.536003, -9.008959),

(-4.444087, 59.652481, -9.474168),

(-3.790154, 60.368027, -33.141968),

],

8: [

(-84.240318, -93.982300, -42.145866),

(-84.631493, -81.721809, -62.446293),

(-95.536392, -63.462791, -50.772717),

(-95.426132, -76.102722, -30.716768),

],

9: [

(17.332596, -100.574318, -101.737862),

(33.688660, -109.248177, -87.696045),

(48.505722, -91.858856, -93.602531),

(31.898283, -83.074646, -108.144897),

],

10: [

(63.556126, 21.762814, -70.147224),

(57.937057, 36.326797, -52.909946),

(73.193756, 26.129313, -38.574009),

(79.125328, 11.190691, -55.811108),

],

11: [

(-59.346130, 35.524860, 7.453290),

(-60.184719, 22.454216, 27.288498),

(-39.490410, 31.872684, 34.366180),

(-38.526440, 44.799294, 14.489631),

],

12: [

(13.395592, -130.138153, 17.346910),

(14.096495, -117.101807, 37.554569),

(-9.711566, -117.223640, 38.739056),

(-10.607868, -129.851654, 18.650393),

],

13: [

(-55.468987, -86.054482, -95.340752),

(-41.909348, -104.651047, -89.491821),

(-24.806488, -96.184753, -103.995155),

(-38.395309, -78.025780, -109.547096),

],

14: [

(47.794785, -33.736324, 62.706944),

(54.499645, -55.719994, 57.413883),

(72.290512, -47.501602, 43.327240),

(65.765244, -25.135725, 48.949913),

],

15: [

(8.913071, -11.333997, 71.133804),

(10.359955, 10.567723, 61.576782),

(-13.275176, 11.878011, 60.495529),

(-14.565926, -9.670244, 70.039528),

],

16: [

(94.452919, -26.296440, 3.009939),

(97.770889, -45.311832, -9.167642),

(98.716286, -32.292400, -28.641371),

(95.505562, -13.785086, -16.689131),

],

17: [

(17.131052, 20.157579, -102.565445),

(-6.934715, 21.642698, -103.673477),

(17.212267, 37.056286, -86.466805),

(-6.426784, 38.582073, -87.356758),

],

18: [

(-100.379051, -24.596079, -27.925400),

(-95.101295, -4.990476, -15.904460),

(-94.848122, -17.101038, 3.973490),

(-100.176735, -36.698360, -7.504705),

],

19: [

(11.144756, -89.171822, 61.853779),

(11.901032, -67.519920, 70.827507),

(-11.875987, -67.272583, 71.388901),

(-12.251099, -88.847527, 62.039257),

],

20: [

(74.651550, -47.856739, -88.944504),

(80.329964, -26.437366, -80.978569),

(65.350769, -16.884157, -95.762360),

(59.673244, -37.826649, -103.784119),

],

21: [

(-78.107231, -8.122915, -81.788292),

(-76.086319, -29.398811, -91.424240),

(-58.610973, -21.703867, -105.075928),

(-60.701736, -0.631173, -95.601372),

],

22: [

(-62.506748, -89.005287, 37.923431),

(-79.309814, -80.821815, 23.486219),

(-75.486038, -99.111618, 8.830044),

(-59.027672, -106.932281, 22.469997),

],

23: [

(52.919292, -12.201375, -104.020058),

(31.645086, -5.642805, -112.863571),

(54.101307, 8.835752, -92.888824),

(33.277386, 15.040869, -101.084915),

],

24: [

(96.137787, -22.737570, -40.440311),

(94.737206, -43.868622, -53.058949),

(86.674965, -31.398907, -71.658997),

(88.221107, -10.872539, -59.398010),

],

25: [

(52.919292, -12.201375, -104.020058),

(31.645086, -5.642805, -112.863571),

(54.101307, 8.835752, -92.888824),

(33.277386, 15.040869, -101.084915),

],

26: [

(-46.220802, 17.946789, -93.364326),

(-47.426178, -2.015542, -105.424500),

(-24.836830, 1.128972, -113.360397),

(-23.782251, 21.463802, -101.025330),

],

27: [

(-87.581352, -2.109334, -60.032719),

(-87.656822, -21.657373, -72.809067),

(-97.336510, -33.203995, -54.061668),

(-96.723816, -13.697992, -41.597427),

],

28: [

(-97.278107, -43.781170, 5.450747),

(-87.839890, -55.416100, 23.972868),

(-88.091766, -75.026360, 11.781352),

(-97.558838, -63.706417, -6.775037),

],

29: [

(-50.173115, -96.246407, 42.043575),

(-27.131416, -102.231842, 49.352020),

(-26.967930, -82.610062, 62.400009),

(-48.840553, -77.042778, 55.464935),

],

30: [

(22.044161, -105.437614, 47.723633),

(23.471203, -86.017601, 60.928928),

(46.141640, -81.779678, 53.330608),

(44.946247, -101.343735, 39.983902),

],

31: [

(83.180389, -82.076492, 9.192251),

(92.816948, -71.009544, -9.696101),

(94.614914, -50.840626, 2.999046),

(84.678734, -62.177277, 22.044226),

],

32: [

(74.998459, 25.100733, -31.993847),

(83.126785, 13.516744, -12.257856),

(69.457230, 27.340242, 1.333983),

(62.081490, 38.044926, -16.917921),

],

33: [

(45.385319, 48.857124, -33.655151),

(50.695263, 40.715889, -54.885075),

(29.824827, 48.044006, -62.718510),

(24.527361, 56.138111, -42.162579),

],

34: [

(-35.592972, 49.306049, -56.470600),

(-12.550899, 53.023766, -62.421768),

(-7.855644, 59.342144, -40.466877),

(-30.697033, 56.044151, -34.455627),

],

35: [

(-79.698730, 21.891609, -15.113928),

(-65.258324, 33.755249, -0.580427),

(-55.139435, 44.713997, -18.927176),

(-69.814827, 33.144806, -33.988605),

],

36: [

(-73.044891, -15.084323, 40.507858),

(-57.852795, 2.089274, 46.474255),

(-66.096619, 15.459711, 27.939346),

(-81.329269, -1.715609, 22.075829),

],

37: [

(-19.522158, -18.200855, 71.464897),

(-26.615257, -40.933121, 72.285690),

(-47.735882, -34.118469, 63.855579),

(-40.733292, -12.025814, 63.028637),

],

38: [

(41.447392, -37.183956, 66.196678),

(36.949070, -13.286051, 64.338356),

(14.734180, -17.359976, 72.015625),

(19.030167, -40.855740, 73.558739),

],

39: [

(73.516617, -24.890842, 41.093018),

(61.766960, -4.614069, 45.573612),

(84.226517, -14.638402, 22.324677),

(72.526840, 5.908772, 26.573786),

],

40: [

(52.602810, -90.789436, -91.095177),

(61.592209, -101.581505, -71.617294),

(76.949615, -83.270264, -68.968239),

(67.921783, -72.666252, -88.338470),

],

41: [

(11.581069, -74.757111, -117.385429),

(32.683720, -81.114250, -108.835655),

(41.108135, -59.303829, -113.232925),

(19.773939, -52.708992, -121.361748),

],

42: [

(-34.558483, -75.891556, -111.963135),

(-39.442699, -53.252758, -116.363701),

(-16.748026, -49.913834, -123.160126),

(-11.876406, -72.864899, -118.576988),

],

43: [

(-67.801003, -99.917587, -70.457497),

(-82.500954, -81.262840, -67.664955),

(-73.283089, -71.476395, -87.192154),

(-58.777195, -90.359016, -89.808731),

],

44: [

(-72.756012, -110.323715, -19.033438),

(-58.137112, -122.528015, -33.683083),

(-67.176285, -111.109970, -52.104473),

(-81.550133, -99.118042, -37.644184),

],

45: [

(-14.956665, -131.241913, 14.224063),

(-8.366419, -138.841125, -7.247497),

(-30.994921, -135.618301, -14.930443),

(-37.410923, -128.030029, 6.694785),

],

46: [

(30.036440, -134.801315, -16.511658),

(37.257572, -127.558159, 5.197520),

(14.987260, -131.575623, 13.573352),

(8.232650, -138.362869, -8.038544),

],

47: [

(77.061188, -100.399788, -39.722916),

(62.263683, -110.834671, -55.276505),

(52.588020, -123.018372, -37.496601),

(67.233017, -112.801338, -22.233381),

],

48: [

(-5.684114, 49.915382, 20.506521),

(17.947041, 47.612324, 21.199146),

```
        (15.914953, 34.067337, 41.175003),
        (-7.372869, 36.764755, 40.290882),
    ],
    49: [
        (-17.293953, -131.369034, -62.358402),
        (6.085686, -131.937653, -63.380001),
        (5.758589, -120.723358, -83.328583),
        (-17.717392, -119.821465, -82.541344),
    ],
}

# ============================================================================
# 7) Compute Unit Vectors from 3D Scan Data
# ============================================================================
def compute_marker_unit_vectors():
    """
    Compute sphere-projected unit vectors for each marker.

    For each marker:
    1. Calculate raw center from 4 corners: p = (c1 + c2 + c3 + c4) / 4
    2. Compute direction from sphere center: u = (p - C) / |p - C|
    3. Store unit vector for pose estimation
    """
    unit_vectors = {}
```

```
    for marker_id, corners in MARKER_CORNERS_RAW.items():
        # Calculate raw center from 4 corners
        corners_np = np.array(corners, dtype=np.float64)
        raw_center = np.mean(corners_np, axis=0)

        # Compute direction vector from sphere center to marker center
        direction = raw_center - SPHERE_CENTER_SCAN
        distance = np.linalg.norm(direction)

        # Normalize to get unit vector
        if distance > 1e-9:
            unit_vector = direction / distance
            unit_vectors[marker_id] = unit_vector

            # Debug: print projection error (should be close to radius)
            # print(f"[DEBUG] ID {marker_id}: distance={distance:.3f}mm,
expected={SPHERE_RADIUS_SCAN:.3f}mm, error={abs(distance-SPHERE_RADIUS_SCAN):.3f}mm",
file=sys.stderr)

    return unit_vectors

# Pre-compute unit vectors at module load
SPHERE_UNIT_VECS = compute_marker_unit_vectors()
```

```
# For sphere center dynamic estimation (marker IDs near center of visible region)




# ============================================================================
# 7.1) Marker Hysteresis Manager
# ============================================================================
class MarkerHysteresis:
    """
    Manages marker state with hysteresis to prevent jitter.

    A marker transitions from INACTIVE -> ACTIVE after being detected
    for HYST_FRAMES_TO_ADD consecutive frames.

    A marker transitions from ACTIVE -> INACTIVE after being missing
    for HYST_FRAMES_TO_REMOVE consecutive frames.

    This prevents rapid on/off switching when a marker is at detection threshold.
    """

    def __init__(self, frames_to_add=3, frames_to_remove=5):
        self.frames_to_add = frames_to_add
        self.frames_to_remove = frames_to_remove
```

```
    # Track consecutive detection/missing counts per marker
    self.detect_count = {}   # mid -> consecutive frames detected
    self.missing_count = {}  # mid -> consecutive frames missing
    self.active_markers = set()  # Currently active (stable) markers

def update(self, detected_ids):
    """
    Update marker states based on current frame's detections.

    Args:
        detected_ids: set or list of marker IDs detected this frame

    Returns:
        set of marker IDs that should be used for pose estimation
    """
    detected_set = set(detected_ids)

    # Process all known markers + newly detected ones
    all_markers = self.active_markers | detected_set | set(self.detect_count.keys())

    for mid in all_markers:
        if mid in detected_set:
            # Marker detected this frame
            self.detect_count[mid] = self.detect_count.get(mid, 0) + 1
```

```
            self.missing_count[mid] = 0

            # Check if should become active
            if mid not in self.active_markers:
                if self.detect_count[mid] >= self.frames_to_add:
                    self.active_markers.add(mid)
                    # print(f"[HYST] Marker {mid} -> ACTIVE", file=sys.stderr)
        else:
            # Marker NOT detected this frame
            self.missing_count[mid] = self.missing_count.get(mid, 0) + 1
            self.detect_count[mid] = 0

            # Check if should become inactive
            if mid in self.active_markers:
                if self.missing_count[mid] >= self.frames_to_remove:
                    self.active_markers.discard(mid)
                    # print(f"[HYST] Marker {mid} -> INACTIVE", file=sys.stderr)

        # Clean up old entries to prevent memory growth
        stale_markers = [mid for mid in self.missing_count
                         if self.missing_count[mid] > self.frames_to_remove * 2
                         and mid not in self.active_markers]
        for mid in stale_markers:
            del self.detect_count[mid]
```

```
            del self.missing_count[mid]

        return self.active_markers.copy()

    def get_filtered_centers(self, centers_dict):
        """
        Filter detected centers to only include active (stable) markers.

        Args:
            centers_dict: {marker_id: (x, y)} from current detection

        Returns:
            Filtered dict with only active markers
        """
        # First update state
        active = self.update(centers_dict.keys())

        # Return only active markers that are also detected this frame
        # (we use current position, but only for stable markers)
        return {mid: centers_dict[mid] for mid in active if mid in centers_dict}

# ============================================================================
# 8) Visualization
# ============================================================================
```

```
DISPLAY_WINDOW = True

DEBUG_SAVE_EVERY_N = 0

DRAW_AXIS_TRIAD = True

AXIS_SCALE = 0.5


# Output to stdout for UI (every N frames)

STDOUT_OUTPUT_EVERY_N = 3


# ============================================================================

# 9) Home Warning

# ============================================================================

HOME_WARN_ENABLED = False

HOME_WARN_MIN_MARKERS = 4

HOME_SHIFT_WARN_PX = 90.0

HOME_SHIFT_CLEAR_PX = 60.0

HOME_WARN_CONSEC_FRAMES = 8

HOME_CLEAR_CONSEC_FRAMES = 15


# ============================================================================

# 10) Calibration

# ============================================================================

def load_calibration(path):

   with open(path, "r") as f:

      data = json.load(f)
```

```
    K = np.array(data["K"], dtype=np.float64)
    dist = np.array(data.get("dist", [0, 0, 0, 0, 0]), dtype=np.float64).reshape(-1, 1)
    imgSize = data.get("imageSize", {})
    calib_w = float(imgSize.get("width", CAPTURE_WIDTH))
    calib_h = float(imgSize.get("height", CAPTURE_HEIGHT))
    sx = CAPTURE_WIDTH / calib_w
    sy = CAPTURE_HEIGHT / calib_h
    K2 = K.copy()
    K2[0, 0] *= sx
    K2[1, 1] *= sy
    K2[0, 2] *= sx
    K2[1, 2] *= sy
    return K2, dist


# ============================================================================
# 11) Camera
# ============================================================================
def start_camera():
    cmd = [
        "rpicam-vid",
        "--width", str(CAPTURE_WIDTH),
        "--height", str(CAPTURE_HEIGHT),
        "--framerate", str(FRAMERATE),
        "--timeout", "0",
```

```
        "--shutter", str(SHUTTER),
        "--gain", str(GAIN),
        "--nopreview",
        "-o", "-",
        "--codec", "yuv420",
    ]
    print(f"[CAMERA] shutter={SHUTTER}, gain={GAIN}", file=sys.stderr)
    return subprocess.Popen(cmd, stdout=subprocess.PIPE, stderr=subprocess.DEVNULL)


def read_frame(proc):
    frame_size = CAPTURE_WIDTH * CAPTURE_HEIGHT * 3 // 2
    raw = proc.stdout.read(frame_size)
    if len(raw) != frame_size:
        return None
    yuv = np.frombuffer(raw, dtype=np.uint8).reshape((CAPTURE_HEIGHT * 3 // 2, CAPTURE_WIDTH))
    bgr = cv2.cvtColor(yuv, cv2.COLOR_YUV2BGR_I420)
    return bgr


# ============================================================================
# 12) ArUco Detection
# ============================================================================
def setup_aruco():
    dictionary = cv2.aruco.Dictionary_get(ARUCO_DICT_TYPE)
    params = cv2.aruco.DetectorParameters_create()
```

```
    params.adaptiveThreshWinSizeMin = 3
    params.adaptiveThreshWinSizeMax = 41
    params.adaptiveThreshWinSizeStep = 6
    params.adaptiveThreshConstant = 7
    params.minMarkerPerimeterRate = 0.008
    params.maxMarkerPerimeterRate = 4.0
    params.polygonalApproxAccuracyRate = 0.06
    params.minCornerDistanceRate = 0.01
    params.minDistanceToBorder = 1

    if USE_CORNER_REFINEMENT:
        try:
            params.cornerRefinementMethod = cv2.aruco.CORNER_REFINE_SUBPIX
            params.cornerRefinementWinSize = 3
            params.cornerRefinementMaxIterations = 15
            params.cornerRefinementMinAccuracy = 0.1
        except AttributeError:
            pass

    return dictionary, params


_clahe = None
```

```
def detect_markers(frame_bgr, dictionary, params):
    global _clahe

    x, y, w, h = ROI_X, ROI_Y, ROI_W, ROI_H
    roi_bgr = frame_bgr[y:y+h, x:x+w]
    gray = cv2.cvtColor(roi_bgr, cv2.COLOR_BGR2GRAY)

    if USE_CLAHE:
        if _clahe is None:
            _clahe = cv2.createCLAHE(clipLimit=CLAHE_CLIP_LIMIT, tileGridSize=CLAHE_TILE_SIZE)
        gray = _clahe.apply(gray)

    corners, ids, rejected = cv2.aruco.detectMarkers(gray, dictionary, parameters=params)

    if corners:
        corners = [c + np.array([x, y], dtype=np.float32) for c in corners]

    return corners, ids, rejected

def marker_centers_px(corners, ids):
    centers = {}
    if ids is None or corners is None:
        return centers
    for i, mid in enumerate(ids.flatten()):
```

```
        c = corners[i][0].mean(axis=0)
        centers[int(mid)] = c.astype(np.float64)
    return centers


# ============================================================================
# 13) Geometry
# ============================================================================
def pixel_to_ray_undist(u, v, K, dist):
    pts = np.array([[[u, v]]], dtype=np.float64)
    und = cv2.undistortPoints(pts, K, dist)
    x = float(und[0, 0, 0])
    y = float(und[0, 0, 1])
    ray = np.array([x, y, 1.0], dtype=np.float64)
    ray /= np.linalg.norm(ray)
    return ray


def sphere_center_cam(K, dist, center_px):
    fx = float(K[0, 0])
    Z = fx * SPHERE_RADIUS_M / float(SPHERE_RADIUS_PX)
    ray = pixel_to_ray_undist(center_px[0], center_px[1], K, dist)
    C = ray * (Z / ray[2])
    return C


def ray_sphere_intersection(ray, C, r):
```

```
    b = -2.0 * float(np.dot(ray, C))
    c = float(np.dot(C, C) - r*r)
    disc = b*b - 4.0*c
    if disc <= 0:
        return None
    s = sqrt(disc)
    t1 = (-b - s) / 2.0
    t2 = (-b + s) / 2.0
    t = None
    for cand in (t1, t2):
        if cand > 0 and (t is None or cand < t):
            t = cand
    if t is None:
        return None
    return t * ray


# ============================================================================
# 14) Rotation
# ============================================================================
def kabsch_rotation(V_from, V_to):
    """Compute optimal rotation using Kabsch algorithm."""
    H = V_from.T @ V_to
    U, S, Vt = np.linalg.svd(H)
    R = Vt.T @ U.T
```

```
    if np.linalg.det(R) < 0:
        Vt[-1, :] *= -1
        R = Vt.T @ U.T
    return R


def rot_to_euler_zyx(R):
    """Convert rotation matrix to Euler angles (ZYX convention)."""
    sy = sqrt(R[0,0]*R[0,0] + R[1,0]*R[1,0])
    singular = sy < 1e-6
    if not singular:
        yaw   = atan2(R[1,0], R[0,0])
        pitch = atan2(-R[2,0], sy)
        roll  = atan2(R[2,1], R[2,2])
    else:
        yaw   = atan2(-R[0,1], R[1,1])
        pitch = atan2(-R[2,0], sy)
        roll  = 0.0
    return np.degrees([roll, pitch, yaw])


# ============================================================================
# 15) Visualization
# ============================================================================
def project_cam_points(points_cam, K, dist):
    pts = np.asarray(points_cam, dtype=np.float64).reshape(-1, 1, 3)
```

```
    rvec = np.zeros((3, 1), dtype=np.float64)
    tvec = np.zeros((3, 1), dtype=np.float64)
    imgpts, _ = cv2.projectPoints(pts, rvec, tvec, K, dist)
    return imgpts.reshape(-1, 2)


def draw_axis_triad(vis, center_px, C_cam, R_cam_sph, K, dist, r_m):
    if R_cam_sph is None:
        return
    pX_s = (AXIS_SCALE * r_m) * np.array([1, 0, 0], dtype=np.float64)
    pY_s = (AXIS_SCALE * r_m) * np.array([0, 1, 0], dtype=np.float64)
    pZ_s = (AXIS_SCALE * r_m) * np.array([0, 0, 1], dtype=np.float64)
    pX_c = C_cam + (R_cam_sph @ pX_s)
    pY_c = C_cam + (R_cam_sph @ pY_s)
    pZ_c = C_cam + (R_cam_sph @ pZ_s)
    pts_2d = project_cam_points(np.stack([pX_c, pY_c, pZ_c], axis=0), K, dist)
    pX_u, pY_u, pZ_u = pts_2d

    cx, cy = int(center_px[0]), int(center_px[1])
    cv2.arrowedLine(vis, (cx, cy), (int(pX_u[0]), int(pX_u[1])), (0, 0, 255), 3, tipLength=0.15)
    cv2.arrowedLine(vis, (cx, cy), (int(pY_u[0]), int(pY_u[1])), (0, 255, 0), 3, tipLength=0.15)
    cv2.arrowedLine(vis, (cx, cy), (int(pZ_u[0]), int(pZ_u[1])), (255, 0, 0), 3, tipLength=0.15)
    cv2.putText(vis, "X", (int(pX_u[0]) + 6, int(pX_u[1]) + 6), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0,
255), 2)
```

```
    cv2.putText(vis, "Y", (int(pY_u[0]) + 6, int(pY_u[1]) + 6), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

    cv2.putText(vis, "Z", (int(pZ_u[0]) + 6, int(pZ_u[1]) + 6), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 0, 0), 2)



# ============================================================================

# 16) UDP Output - Sends to Panel (localhost) and MATLAB PC

# ============================================================================

class UDPSender:

    """

    Sends pose data over UDP to multiple targets.

    """



    def __init__(self, target_ip, target_port, target_ip_2=None, target_port_2=None, send_rate_hz=30):

        self.targets = [(target_ip, target_port)]

        if target_ip_2 and target_port_2:

            self.targets.append((target_ip_2, target_port_2))



        self.min_interval = 1.0 / send_rate_hz if send_rate_hz > 0 else 0

        self.last_send_time = 0



        self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

        self.sock.setblocking(False)

        self.packet_count = 0
```

```
def should_send(self):
    if self.min_interval == 0:
        return True
    return (time.time() - self.last_send_time) >= self.min_interval

def send_pose(self, roll, pitch, yaw, timestamp=None):
    """
    Send pose data as binary packet.

    Args:
        roll, pitch, yaw: Angles in degrees (can be nan for invalid)
        timestamp: Unix timestamp (default: current time)
    """
    if not self.should_send():
        return False

    if timestamp is None:
        timestamp = time.time()

    # Check validity
    valid = 1 if (not np.isnan(roll) and not np.isnan(pitch) and not np.isnan(yaw)) else 0

    # Replace nan with 0 for transmission
```

```
    r = roll if not np.isnan(roll) else 0.0
    p = pitch if not np.isnan(pitch) else 0.0
    y = yaw if not np.isnan(yaw) else 0.0

    # Pack as binary: 4 doubles + 1 uint32
    # Format: '<' = little-endian, 'd' = double, 'I' = unsigned int
    packet = struct.pack('<ddddI', timestamp, r, p, y, valid)

    try:
        self.sock.sendto(packet, (self.target_ip, self.target_port))
        self.last_send_time = time.time()
        self.packet_count += 1
        return True
    except Exception as e:
        # Non-blocking socket might fail if buffer full - ignore
        return False

def send_pose_json(self, roll, pitch, yaw, num_detected=0, num_used=0, fps=0.0, timestamp=None):
    """
    Send pose data as JSON string to all targets.
    """
    if not self.should_send():
        return False
```

```
if timestamp is None:
    timestamp = time.time()

valid = not (np.isnan(roll) or np.isnan(pitch) or np.isnan(yaw))

data = {
    "roll": round(roll, 2) if valid else None,
    "pitch": round(pitch, 2) if valid else None,
    "yaw": round(yaw, 2) if valid else None,
    "detected": num_detected,
    "used": num_used,
    "fps": round(fps, 1)
}

packet = json.dumps(data).encode('utf-8')

try:
    for target_ip, target_port in self.targets:
        self.sock.sendto(packet, (target_ip, target_port))
    self.last_send_time = time.time()
    self.packet_count += 1
    return True
except:
    return False
```

```
    def close(self):
        self.sock.close()


# ============================================================================
# 17) STDOUT Output for UI
# ============================================================================
def output_to_ui(fps, num_detected, num_used, roll, pitch, yaw):
    """Output JSON data to stdout for UI consumption."""
    data = {
        "fps": round(fps, 1),
        "detected": num_detected,
        "used": num_used,
        "roll": round(roll, 2) if not np.isnan(roll) else None,
        "pitch": round(pitch, 2) if not np.isnan(pitch) else None,
        "yaw": round(yaw, 2) if not np.isnan(yaw) else None
    }
    print(json.dumps(data), flush=True)


# ============================================================================
# 17) MAIN
# ============================================================================
def main():
    os.makedirs(OUTPUT_DIR, exist_ok=True)
```

```
    if not os.path.exists(CALIB_FILE):
        raise FileNotFoundError(f"{CALIB_FILE} not found.")

    K, dist = load_calibration(CALIB_FILE)
    print(f"[OK] Calibration loaded.", file=sys.stderr)
    print(f"[V5] Using 3D scan coordinates - {len(SPHERE_UNIT_VECS)} markers with unit vectors",
file=sys.stderr)
    print(f"[V5] Sphere center (scan): {SPHERE_CENTER_SCAN}", file=sys.stderr)
    print(f"[V5] Sphere radius (scan): {SPHERE_RADIUS_SCAN:.3f}mm =
{SPHERE_RADIUS_M*1000:.3f}mm (tracking)", file=sys.stderr)
    print(f"[CONFIG] CLAHE: {USE_CLAHE}, Corner Refinement: {USE_CORNER_REFINEMENT}",
file=sys.stderr)
    print(f"[CONFIG] Center: {SPHERE_CENTER_PX_FALLBACK}, Radius: {SPHERE_RADIUS_PX}px",
file=sys.stderr)
    print(f"[CONFIG] ROI: ({ROI_X}, {ROI_Y}, {ROI_W}, {ROI_H})", file=sys.stderr)
    print(f"[CONFIG] Hysteresis: {HYSTERESIS_ENABLED} (add={HYST_FRAMES_TO_ADD},
remove={HYST_FRAMES_TO_REMOVE})", file=sys.stderr)
    print(f"[CONFIG] Smoothing: {SMOOTHING_ENABLED} (alpha={SMOOTHING_ALPHA})",
file=sys.stderr)
    print(f"[CONFIG] Radius Limit: {RADIUS_LIMIT_ENABLED} (fraction={RADIUS_LIMIT_FRACTION},
max_px={int(SPHERE_RADIUS_PX * RADIUS_LIMIT_FRACTION)})", file=sys.stderr)
    print(f"[CONFIG] UDP Output: {UDP_OUTPUT_ENABLED}
(target={UDP_TARGET_IP}:{UDP_TARGET_PORT}, rate={UDP_SEND_RATE_HZ}Hz)", file=sys.stderr)

    dictionary, params = setup_aruco()
```

```
    cam = start_camera()

    time.sleep(1.0)


    home_warn_on = False

    home_warn_count = 0

    home_clear_count = 0


    # Initialize marker hysteresis manager

    marker_hyst = MarkerHysteresis(HYST_FRAMES_TO_ADD, HYST_FRAMES_TO_REMOVE) if
HYSTERESIS_ENABLED else None


    # Initialize UDP sender - sends to both Panel (localhost) and MATLAB PC

    udp_sender = UDPSender(UDP_TARGET_IP, UDP_TARGET_PORT,

                           UDP_TARGET_IP_2, UDP_TARGET_PORT_2,

                           UDP_SEND_RATE_HZ) if UDP_OUTPUT_ENABLED else None

    if udp_sender:

        print(f"[UDP] Target 1: {UDP_TARGET_IP}:{UDP_TARGET_PORT} (Panel)", file=sys.stderr)

        print(f"[UDP] Target 2: {UDP_TARGET_IP_2}:{UDP_TARGET_PORT_2} (MATLAB)", file=sys.stderr)


    with open(CSV_FILE, "w", newline="") as fcsv:

        writer = csv.writer(fcsv)

        writer.writerow([

            "timestamp_s", "frame",

            "num_detected", "num_used",
```

```
    "roll_deg", "pitch_deg", "yaw_deg",
    "center_x", "center_y",
    "home_shift_px", "home_warn"
])

frame_idx = 0
fps_cnt = 0
fps_t = time.time()
fps = 0.0

# Smoothing state (EMA filter)
smooth_roll = None
smooth_pitch = None
smooth_yaw = None

try:
    while True:
        frame = read_frame(cam)
        if frame is None:
            continue
        frame_idx += 1

        fps_cnt += 1
        dt = time.time() - fps_t
```

```
        if dt >= 1.0:
            fps = fps_cnt / dt
            fps_cnt = 0
            fps_t = time.time()

        corners, ids, rejected = detect_markers(frame, dictionary, params)
        centers = marker_centers_px(corners, ids)

        center_px = SPHERE_CENTER_PX_FALLBACK

        home_shift_px = np.nan
        if HOME_WARN_ENABLED and len(centers) >= HOME_WARN_MIN_MARKERS:
            mean_all = np.mean(np.stack(list(centers.values()), axis=0), axis=0)
            home_shift_px = float(np.linalg.norm(mean_all - np.array(SPHERE_CENTER_PX_FALLBACK,
dtype=np.float64)))
            if home_shift_px >= HOME_SHIFT_WARN_PX:
                home_warn_count += 1
                home_clear_count = 0
            elif home_shift_px <= HOME_SHIFT_CLEAR_PX:
                home_clear_count += 1
                home_warn_count = 0
            if (not home_warn_on) and home_warn_count >= HOME_WARN_CONSEC_FRAMES:
                home_warn_on = True
                print(f"[WARN] Sphere moved. shift_px={home_shift_px:.1f}", file=sys.stderr)
```

```
    if home_warn_on and home_clear_count >= HOME_CLEAR_CONSEC_FRAMES:
        home_warn_on = False
        print(f"[INFO] Sphere back. shift_px={home_shift_px:.1f}", file=sys.stderr)

C = sphere_center_cam(K, dist, center_px)

# Apply hysteresis filter to stabilize marker set
if marker_hyst is not None:
    stable_centers = marker_hyst.get_filtered_centers(centers)
else:
    stable_centers = centers

# Apply radius limit - only use markers within trusted zone
if RADIUS_LIMIT_ENABLED:
    max_dist_px = SPHERE_RADIUS_PX * RADIUS_LIMIT_FRACTION
    filtered_centers = {}
    for mid, (u, v) in stable_centers.items():
        dist_from_center = np.sqrt((u - center_px[0])**2 + (v - center_px[1])**2)
        if dist_from_center <= max_dist_px:
            filtered_centers[mid] = (u, v)
    centers_for_pose = filtered_centers
else:
    centers_for_pose = stable_centers
```

```
V_sph = []

V_cam = []

used_marker_ids = []

for mid, (u, v) in centers_for_pose.items():

  if mid not in SPHERE_UNIT_VECS:

    continue

  ray = pixel_to_ray_undist(u, v, K, dist)

  P = ray_sphere_intersection(ray, C, SPHERE_RADIUS_M)

  if P is None:

    continue

  v_cam = (P - C) / SPHERE_RADIUS_M

  n = np.linalg.norm(v_cam)

  if n < 1e-9:

    continue

  v_cam /= n

  V_sph.append(SPHERE_UNIT_VECS[mid])

  V_cam.append(v_cam)

  used_marker_ids.append(mid)


num_detected = len(centers)        # Raw detection count

num_stable = len(stable_centers)    # After hysteresis filter

num_in_zone = len(centers_for_pose)  # Within radius limit

num_used = len(V_cam)              # Actually used for pose
```

```
R_cam_sph = None
roll_raw = pitch_raw = yaw_raw = np.nan
roll = pitch = yaw = np.nan

if num_used >= 3:
    V_sph_np = np.stack(V_sph, axis=0)
    V_cam_np = np.stack(V_cam, axis=0)
    R_cam_sph = kabsch_rotation(V_sph_np, V_cam_np)
    roll_raw, pitch_raw, yaw_raw = rot_to_euler_zyx(R_cam_sph)

    # Apply temporal smoothing (EMA filter)
    if SMOOTHING_ENABLED:
        alpha = SMOOTHING_ALPHA
        if smooth_roll is None:
            # First valid measurement - initialize
            smooth_roll = roll_raw
            smooth_pitch = pitch_raw
            smooth_yaw = yaw_raw
        else:
            # Handle angle wrapping for yaw (can jump between -180 and 180)
            # Use circular mean for angles
            def smooth_angle(prev, new, a):
                diff = new - prev
                # Wrap difference to [-180, 180]
```

```
            while diff > 180: diff -= 360
            while diff < -180: diff += 360
            return prev + a * diff

        smooth_roll = smooth_angle(smooth_roll, roll_raw, alpha)
        smooth_pitch = smooth_angle(smooth_pitch, pitch_raw, alpha)
        smooth_yaw = smooth_angle(smooth_yaw, yaw_raw, alpha)

      roll, pitch, yaw = smooth_roll, smooth_pitch, smooth_yaw
   else:
      roll, pitch, yaw = roll_raw, pitch_raw, yaw_raw

# Output to UI (every N frames to reduce overhead)
if frame_idx % STDOUT_OUTPUT_EVERY_N == 0:
   output_to_ui(fps, num_detected, num_used, roll, pitch, yaw)

# Send pose data via UDP (JSON format for Panel)
if udp_sender:
   udp_sender.send_pose_json(roll, pitch, yaw, num_detected, num_used, fps)

ts = time.time()
writer.writerow([
   ts, frame_idx, num_detected, num_used,
   roll, pitch, yaw,
```

```
    center_px[0], center_px[1],
    home_shift_px, int(home_warn_on)
])
fcsv.flush()

# Visualization
vis = frame.copy()
cv2.rectangle(vis, (ROI_X, ROI_Y), (ROI_X + ROI_W, ROI_Y + ROI_H), (0, 255, 255), 2)
cv2.circle(vis, (int(center_px[0]), int(center_px[1])), 5, (0, 255, 255), -1)
cv2.circle(vis, (int(center_px[0]), int(center_px[1])), SPHERE_RADIUS_PX, (0, 255, 255), 2)

# Draw radius limit zone (green circle)
if RADIUS_LIMIT_ENABLED:
    limit_radius = int(SPHERE_RADIUS_PX * RADIUS_LIMIT_FRACTION)
    cv2.circle(vis, (int(center_px[0]), int(center_px[1])), limit_radius, (0, 255, 0), 1)

if corners is not None and ids is not None and len(corners) > 0:
    cv2.aruco.drawDetectedMarkers(vis, corners, ids)
    for mid, cxy in centers.items():
        # Color coding:
        # Green = used for pose (stable + in zone)
        # Cyan = stable but outside zone
        # Orange = detected but not stable yet
        if mid in centers_for_pose:
```

```
                color = (0, 255, 0)    # Green - used for pose
            elif mid in stable_centers:
                color = (255, 255, 0)  # Cyan - stable but outside radius
            else:
                color = (0, 165, 255)  # Orange - not stable yet
            cv2.circle(vis, (int(cxy[0]), int(cxy[1])), 4, color, -1)
            cv2.putText(vis, str(mid), (int(cxy[0]) + 8, int(cxy[1])),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 2)

        y0 = 25
        cv2.putText(vis, f"FPS: {fps:.1f}", (10, y0), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
        y0 += 28

        # Show detected, stable, in-zone, and used counts
        if RADIUS_LIMIT_ENABLED:
            cv2.putText(vis, f"Det:{num_detected} Stb:{num_stable} Zone:{num_in_zone}
Used:{num_used}", (10, y0),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)
        elif HYSTERESIS_ENABLED:
            cv2.putText(vis, f"Det: {num_detected}  Stable: {num_stable}  Used: {num_used}", (10, y0),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
        else:
            cv2.putText(vis, f"Detected: {num_detected}  Used(3D): {num_used}", (10, y0),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
```

```
y0 += 28

# V5 indicator with filter status
filters = []
if HYSTERESIS_ENABLED: filters.append(f"Hyst")
if SMOOTHING_ENABLED: filters.append(f"Sm:{SMOOTHING_ALPHA}")
if RADIUS_LIMIT_ENABLED: filters.append(f"R:{RADIUS_LIMIT_FRACTION}")
filter_str = " +" + "+".join(filters) if filters else ""
cv2.putText(vis, f"V5: 3D Scan{filter_str}", (10, y0),
        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 200, 0), 2)
y0 += 28

if HOME_WARN_ENABLED and not np.isnan(home_shift_px):
   cv2.putText(vis, f"Home shift(px): {home_shift_px:.1f}", (10, y0),
           cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
   y0 += 28

if home_warn_on:
   cv2.putText(vis, "WARNING: SPHERE MOVED!", (10, y0),
           cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 0, 255), 3)
   y0 += 34

if R_cam_sph is not None:
   cv2.putText(vis, f"ABS Pose (deg) roll={roll:.1f} pitch={pitch:.1f} yaw={yaw:.1f}",
```

```
                (10, y0), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 0), 2)
            y0 += 28
        else:
            cv2.putText(vis, "ABS Pose: need >=3 mapped IDs",
                (10, y0), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
            y0 += 28

        if DRAW_AXIS_TRIAD and R_cam_sph is not None:
            draw_axis_triad(vis, center_px, C, R_cam_sph, K, dist, SPHERE_RADIUS_M)

        if DEBUG_SAVE_EVERY_N > 0 and frame_idx % DEBUG_SAVE_EVERY_N == 0:
            cv2.imwrite(os.path.join(OUTPUT_DIR, f"debug_{frame_idx:06d}.jpg"), vis)

        if DISPLAY_WINDOW:
            cv2.imshow("Sphere Tracker V5 - 3D Scan", vis)
            key = cv2.waitKey(1) & 0xFF
            if key == ord('q'):
                break

finally:
    cam.terminate()
    cam.wait()
    cv2.destroyAllWindows()
    if udp_sender:
```

print(f"[UDP] Sent {udp_sender.packet_count} packets", file=sys.stderr)

udp_sender.close()

print(f"[OK] Log saved: {CSV_FILE}", file=sys.stderr)

if __name__ == "__main__":

main()

C. sphere_panel_v3.py

```
#!/usr/bin/env python3
"""
Sphere Panel V3
---------------
Ports:
- 25050: Tracker -> Panel (Camera JSON: roll, pitch, yaw, detected, used, fps)
- 25051: Simulation -> Panel (Encoder JSON: roll, pitch, yaw = camera + drift)

Drift = Encoder - Camera (simülasyon tarafından eklenen)
"""

import tkinter as tk
import subprocess, threading, queue, time, json, os, socket, struct, math
from collections import deque

CAMERA_PORT = 25050
ENCODER_PORT = 25051
```

```
DRIFT_WARN = 6.0

DRIFT_ALARM = 10.0


TRACKER = os.path.join(os.path.dirname(os.path.abspath(__file__)), "sphere_rotation_tracker_V6.py")


C = {"bg":"#0f0f1a", "card":"#1e1e35", "green":"#22c55e", "red":"#ef4444",

    "yellow":"#f59e0b", "cyan":"#06b6d4", "purple":"#6366f1", "txt":"#a0a0b0", "bdr":"#303050"}


class Receiver:

   def __init__(s, q):

      s.q = q

      s.on = False

      s.cam_sock = s.enc_sock = None

      s.cam_rpy = [0,0,0]

      s.enc_rpy = [0,0,0]

      s.cam_t = s.enc_t = 0


   def start(s):

      s.on = True

      try:

         s.cam_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

         s.cam_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

         s.cam_sock.bind(('0.0.0.0', CAMERA_PORT))

         s.cam_sock.settimeout(0.1)
```

```
        s.enc_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        s.enc_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        s.enc_sock.bind(('0.0.0.0', ENCODER_PORT))
        s.enc_sock.settimeout(0.1)
        threading.Thread(target=s._cl, daemon=True).start()
        threading.Thread(target=s._el, daemon=True).start()
        return True
    except Exception as e:
        print(f"[ERR] {e}")
        return False

def stop(s):
    s.on = False
    time.sleep(0.2)
    try:
        if s.cam_sock: s.cam_sock.close()
        if s.enc_sock: s.enc_sock.close()
    except: pass

def _cl(s):
    while s.on:
        try:
            d, _ = s.cam_sock.recvfrom(1024)
            s.cam_t = time.time()
```

```
            j = json.loads(d.decode())
            r = j.get('roll', 0) or 0
            p = j.get('pitch', 0) or 0
            y = j.get('yaw', 0) or 0
            s.cam_rpy = [r, p, y]
            s.q.put(('cam', {'rpy':(r,p,y), 'det':j.get('detected',0), 'used':j.get('used',0), 'fps':j.get('fps',0)}))
            s._dr()
        except: pass


def _el(s):
    while s.on:
        try:
            d, _ = s.enc_sock.recvfrom(1024)
            s.enc_t = time.time()
            try:
                j = json.loads(d.decode())
                r = j.get('roll', 0) or 0
                p = j.get('pitch', 0) or 0
                y = j.get('yaw', 0) or 0
                s.enc_rpy = [r, p, y]
            except:
                if len(d) == 24:
                    vals = struct.unpack('ddd', d)
                    s.enc_rpy = list(vals)
```

```
            s.q.put(('enc', {'rpy':tuple(s.enc_rpy)}))
            s._dr()
        except: pass

    def _dr(s):
        if s.cam_t == 0 or s.enc_t == 0: return
        dr = s.enc_rpy[0] - s.cam_rpy[0]
        dp = s.enc_rpy[1] - s.cam_rpy[1]
        dy = s.enc_rpy[2] - s.cam_rpy[2]
        drift = math.sqrt(dr*dr + dp*dp + dy*dy)
        st = 'alarm' if drift >= DRIFT_ALARM else ('warn' if drift >= DRIFT_WARN else 'ok')
        s.q.put(('drift', {'val':drift, 'dr':dr, 'dp':dp, 'dy':dy, 'st':st}))

    def cam_ok(s): return time.time()-s.cam_t < 1
    def enc_ok(s): return time.time()-s.enc_t < 1

class Graph(tk.Canvas):
    def __init__(s, p, w=175, h=80, t="", c="#22c55e", mn=-90, mx=90):
        super().__init__(p, width=w, height=h, bg=C["card"], highlightthickness=1,
highlightbackground=C["bdr"])
        s.w,s.h,s.t,s.c,s.mn,s.mx = w,h,t,c,mn,mx
        s.d = deque(maxlen=100)
        s._dr()
    def _dr(s):
```

```
        s.delete("all")
        s.create_text(s.w//2, 9, text=s.t, fill=C["txt"], font=("Arial",8))
        if len(s.d)>=2:
            pts=[]
            for i,v in enumerate(s.d):
                x=22+(i/(len(s.d)-1))*(s.w-27)
                y=16+(1-(max(s.mn,min(s.mx,v))-s.mn)/(s.mx-s.mn))*(s.h-21)
                pts.extend([x,y])
            if len(pts)>=4: s.create_line(pts, fill=s.c, width=2, smooth=True)
        if s.d: s.create_text(s.w-4, 9, text=f"{s.d[-1]:.1f}", fill=s.c, font=("Arial",8,"bold"), anchor="e")
    def add(s,v): s.d.append(v); s._dr()
    def clr(s): s.d.clear(); s._dr()


class Drift(tk.Canvas):
    def __init__(s, p, w=160, h=140):
        super().__init__(p, width=w, height=h, bg=C["card"], highlightthickness=1,
highlightbackground=C["bdr"])
        s.w,s.h,s.val,s.dr,s.dp,s.dy,s.st = w,h,0,0,0,0,'?'
        s._dr()
    def _dr(s):
        s.delete("all")
        s.create_text(s.w//2, 10, text="DRIFT", fill=C["txt"], font=("Arial",9))
        clr={'alarm':C["red"],'warn':C["yellow"],'ok':C["green"]}.get(s.st,C["txt"])
        txt={'alarm':"ALARM",'warn':"WARN",'ok':"OK"}.get(s.st,"--")
```

```
        s.create_text(s.w//2, 45, text=f"{s.val:.1f}°", fill=clr, font=("Arial",24,"bold"))
        s.create_text(s.w//2, 75, text=txt, fill=clr, font=("Arial",10,"bold"))
        s.create_text(s.w//2, 95, text=f"R:{s.dr:.1f} P:{s.dp:.1f} Y:{s.dy:.1f}", fill=C["txt"], font=("Arial",8))
    def upd(s,v,dr,dp,dy,st): s.val,s.dr,s.dp,s.dy,s.st=v,dr,dp,dy,st; s._dr()

class Conn(tk.Canvas):
    def __init__(s, p, lbl, w=120, h=26):
        super().__init__(p, width=w, height=h, bg=C["card"], highlightthickness=0)
        s.lbl,s.w,s.h,s.ok = lbl,w,h,False
        s._dr()
    def _dr(s):
        s.delete("all")
        clr=C["green"] if s.ok else C["red"]
        s.create_oval(6,s.h//2-4,14,s.h//2+4,fill=clr,outline=clr)
        s.create_text(20,s.h//2,text=s.lbl,fill=clr,anchor="w",font=("Arial",9))
    def set(s,ok): s.ok=ok; s._dr()

class App:
    def __init__(s):
        s.root=tk.Tk(); s.root.title("Panel V3"); s.root.geometry("900x620"); s.root.configure(bg=C["bg"])
        s.q=queue.Queue(); s.recv=Receiver(s.q); s.ron=False; s.tp=None; s.ton=False; s._ui();
s.root.after(50,s._upd)

    def _ui(s):
```

```
        tk.Label(s.root,text="Sphere Panel
V3",bg=C["bg"],fg=C["cyan"],font=("Arial",14,"bold")).pack(pady=5)

        tk.Label(s.root,text=f"Cam:{CAMERA_PORT}
Enc:{ENCODER_PORT}",bg=C["bg"],fg=C["txt"],font=("Arial",8)).pack()

        m=tk.Frame(s.root,bg=C["bg"]); m.pack(fill=tk.BOTH,expand=True,padx=5)



        l=tk.Frame(m,bg=C["bg"],width=170); l.pack(side=tk.LEFT,fill=tk.Y,padx=(0,5));
l.pack_propagate(False)

        tk.Label(l,text="Receiver",bg=C["card"],fg=C["txt"]).pack(fill=tk.X)

        s.br=tk.Button(l,text="▶Start",command=s._tr,bg=C["cyan"],fg="white",relief="flat");
s.br.pack(fill=tk.X,pady=2)

        tk.Label(l,text="Tracker V6",bg=C["card"],fg=C["txt"]).pack(fill=tk.X,pady=(8,0))

        s.bt=tk.Button(l,text="▶Start",command=s._tt,bg=C["green"],fg="white",relief="flat");
s.bt.pack(fill=tk.X,pady=2)

        s.ts=tk.Label(l,text="●Off",bg=C["bg"],fg=C["red"]); s.ts.pack()

        tk.Label(l,text="Stats",bg=C["card"],fg=C["txt"]).pack(fill=tk.X,pady=(8,0))

        s.sd=tk.Label(l,text="Det:--",bg=C["bg"],fg=C["txt"],font=("Arial",8)); s.sd.pack(anchor="w")

        s.su=tk.Label(l,text="Use:--",bg=C["bg"],fg=C["txt"],font=("Arial",8)); s.su.pack(anchor="w")

        s.sf=tk.Label(l,text="FPS:--",bg=C["bg"],fg=C["txt"],font=("Arial",8)); s.sf.pack(anchor="w")



        c=tk.Frame(m,bg=C["bg"]); c.pack(side=tk.LEFT,fill=tk.BOTH,expand=True,padx=(0,5))

        tk.Label(c,text="Camera (Tracker)",bg=C["card"],fg=C["txt"]).pack(fill=tk.X)

        cf=tk.Frame(c,bg=C["card"]); cf.pack(fill=tk.X,pady=2)

        s.cr=Graph(cf,t="Roll",c="#ef4444",mn=-90,mx=90); s.cr.pack(side=tk.LEFT,padx=1)

        s.cp=Graph(cf,t="Pitch",c="#22c55e",mn=-90,mx=90); s.cp.pack(side=tk.LEFT,padx=1)

        s.cy=Graph(cf,t="Yaw",c="#3b82f6",mn=-180,mx=180); s.cy.pack(side=tk.LEFT,padx=1)
```

```
        tk.Label(c,text="Encoder (Sim = Cam + Drift)",bg=C["card"],fg=C["txt"]).pack(fill=tk.X,pady=(5,0))
        ef=tk.Frame(c,bg=C["card"]); ef.pack(fill=tk.X,pady=2)
        s.er=Graph(ef,t="Roll",c="#f87171",mn=-90,mx=90); s.er.pack(side=tk.LEFT,padx=1)
        s.ep=Graph(ef,t="Pitch",c="#4ade80",mn=-90,mx=90); s.ep.pack(side=tk.LEFT,padx=1)
        s.ey=Graph(ef,t="Yaw",c="#60a5fa",mn=-180,mx=180); s.ey.pack(side=tk.LEFT,padx=1)
        tk.Label(c,text="Drift Over Time",bg=C["card"],fg=C["txt"]).pack(fill=tk.X,pady=(5,0))
        s.dg=Graph(c,w=530,h=70,t="",c=C["yellow"],mn=0,mx=20); s.dg.pack(fill=tk.X,pady=2)

        r=tk.Frame(m,bg=C["bg"],width=175); r.pack(side=tk.RIGHT,fill=tk.Y); r.pack_propagate(False)
        tk.Label(r,text="Connections",bg=C["card"],fg=C["txt"]).pack(fill=tk.X)
        s.cc=Conn(r,"Camera"); s.cc.pack(pady=2)
        s.ec=Conn(r,"Encoder"); s.ec.pack(pady=2)
        tk.Label(r,text="Drift",bg=C["card"],fg=C["txt"]).pack(fill=tk.X,pady=(5,0))
        s.dm=Drift(r); s.dm.pack(pady=3)

    def _tr(s):
        if s.ron: s.recv.stop(); s.ron=False; s.br.config(text="▶Start",bg=C["cyan"]); s.cc.set(False);
s.ec.set(False)
        else:
            if s.recv.start(): s.ron=True; s.br.config(text="⏹Stop",bg=C["red"])

    def _tt(s):
        if s.ton:
            if s.tp: s.tp.terminate()
```

```
            s.tp=None; s.ton=False; s.bt.config(text="▶Start",bg=C["green"]);
s.ts.config(text="●Off",fg=C["red"])

        else:

            if not os.path.exists(TRACKER): print(f"[ERR] {TRACKER} not found"); return

            try:
s.tp=subprocess.Popen(["python3",TRACKER],stdout=subprocess.PIPE,stderr=subprocess.PIPE);
s.ton=True; s.bt.config(text="⏹Stop",bg=C["red"]); s.ts.config(text="●On",fg=C["green"])

            except: pass



    def _upd(s):

        try:

            while True:

                t,d=s.q.get_nowait()

                if t=='cam': r=d.get('rpy',(0,0,0)); s.cr.add(r[0]); s.cp.add(r[1]); s.cy.add(r[2]);
s.sd.config(text=f"Det:{d.get('det','--')}"); s.su.config(text=f"Use:{d.get('used','--')}");
s.sf.config(text=f"FPS:{d.get('fps',0):.1f}")

                elif t=='enc': r=d.get('rpy',(0,0,0)); s.er.add(r[0]); s.ep.add(r[1]); s.ey.add(r[2])

                elif t=='drift': s.dm.upd(d['val'],d['dr'],d['dp'],d['dy'],d['st']); s.dg.add(d['val'])

        except: pass

        if s.ron: s.cc.set(s.recv.cam_ok()); s.ec.set(s.recv.enc_ok())

        s.root.after(50,s._upd)



    def run(s):

        def cl():

            if s.ron: s.recv.stop()

            if s.tp: s.tp.terminate()
```

```
            s.root.destroy()

        s.root.protocol("WM_DELETE_WINDOW",cl); s.root.mainloop()


if __name__=="__main__": App().run()
```

D. sphere_simulationv2.m

```
%% SPHERE SIMULATION V3

% ========================================================================

% Camera Follower + Drift Injection OR Standalone Waveform Generator

%

% TWO MODES:

% 1. FOLLOW CAMERA: Listens to tracker (port 25050), adds drift, sends as encoder

% 2. STANDALONE: Generates waveforms independently (like V2)

%

% Ports:

% - RECEIVE from Tracker: 25050 (JSON: roll, pitch, yaw)

% - SEND to Panel: 25051 (JSON: roll, pitch, yaw)

% ========================================================================


function sphere_simulation_v3()

    close all; clear; clc;


    fprintf('=== Sphere Simulation V3 ===\n');

    fprintf('Camera Follower + Drift Injection\n\n');
```

```
%% CONFIGURATION
TRACKER_PORT = 25050;      % Receive camera data (from Tracker)
PANEL_PORT = 25051;        % Send encoder data (to Panel)
PANEL_IP = '172.20.10.3';  % Pi IP - Panel burda çalışıyor!

SAMPLE_RATE = 30;
DT = 1 / SAMPLE_RATE;

%% CREATE UDP SOCKETS
% Receiver for camera data
try
    cam_socket = udp('0.0.0.0', 'LocalPort', TRACKER_PORT);
    cam_socket.InputBufferSize = 1024;
    cam_socket.Timeout = 0.01;
    fopen(cam_socket);
    fprintf('[UDP] Listening for camera on port %d\n', TRACKER_PORT);
catch e
    fprintf('[ERROR] Camera socket: %s\n', e.message);
    fprintf('Try: delete(instrfindall)\n');
    return;
end

% Sender for encoder data
try
```

```
    enc_socket = udp(PANEL_IP, PANEL_PORT);
    enc_socket.OutputBufferSize = 256;
    fopen(enc_socket);
    fprintf('[UDP] Sending encoder to %s:%d\n', PANEL_IP, PANEL_PORT);
catch e
    fprintf('[ERROR] Encoder socket: %s\n', e.message);
    fclose(cam_socket); delete(cam_socket);
    return;
end

%% CREATE FIGURE
fig = figure('Name', 'Sphere Simulation V3', ...
             'Position', [100 100 900 650], ...
             'Color', [0.1 0.1 0.15], ...
             'NumberTitle', 'off', ...
             'MenuBar', 'none', ...
             'CloseRequestFcn', @onClose);

h = struct();
h.cam_socket = cam_socket;
h.enc_socket = enc_socket;
h.running = true;
h.paused = false;
h.t = 0;
```

```
h.packet_count = 0;


% Camera values

h.cam_roll = 0;

h.cam_pitch = 0;

h.cam_yaw = 0;

h.cam_connected = false;

h.last_cam_time = 0;


% Mode: true = follow camera, false = standalone waveform

h.follow_mode = true;


% Drift (added to camera values)

h.drift_enabled = false;

h.drift_rate = [0, 0, 0];  % deg/s for each axis

h.accum_drift = [0, 0, 0]; % accumulated drift


% Standalone waveform (when not following camera)

h.wave_enabled = [false, false, false];

h.wave_amp = [30, 30, 30];

h.wave_freq = [0.3, 0.2, 0.1];


% Noise

h.noise_enabled = false;
```

```
h.noise_amp = 0.5;


%% TITLE
uicontrol('Style', 'text', 'String', 'SPHERE SIMULATION V3', ...
        'Position', [300 610 300 30], 'FontSize', 14, 'FontWeight', 'bold', ...
        'ForegroundColor', [0.4 0.8 1], 'BackgroundColor', [0.1 0.1 0.15]);


%% MODE PANEL
mode_panel = uipanel('Title', 'MODE', 'Position', [0.02 0.75 0.3 0.18], ...
                'BackgroundColor', [0.15 0.12 0.2], 'ForegroundColor', [0.8 0.8 1], 'FontSize', 10);


h.mode_follow = uicontrol(mode_panel, 'Style', 'radiobutton', ...
                    'String', 'Follow Camera + Drift', ...
                    'Position', [10 70 180 25], 'Value', 1, ...
                    'BackgroundColor', [0.15 0.12 0.2], 'ForegroundColor', [0.5 1 0.5], ...
                    'Callback', @(~,~) setMode(fig, true));


h.mode_standalone = uicontrol(mode_panel, 'Style', 'radiobutton', ...
                       'String', 'Standalone Waveform', ...
                       'Position', [10 40 180 25], 'Value', 0, ...
                       'BackgroundColor', [0.15 0.12 0.2], 'ForegroundColor', [1 0.8 0.5], ...
                       'Callback', @(~,~) setMode(fig, false));


h.cam_status = uicontrol(mode_panel, 'Style', 'text', ...
```

```
                'String', 'Camera: Waiting...', ...
                'Position', [10 10 200 22], ...
                'BackgroundColor', [0.15 0.12 0.2], 'ForegroundColor', [1 0.5 0.5]);

%% CAMERA INPUT DISPLAY
cam_panel = uipanel('Title', 'Camera Input', 'Position', [0.34 0.75 0.3 0.18], ...
            'BackgroundColor', [0.12 0.15 0.12], 'ForegroundColor', [0.5 1 0.5], 'FontSize', 10);

h.cam_roll_lbl = uicontrol(cam_panel, 'Style', 'text', 'String', 'R: 0.0', ...
                 'Position', [10 60 80 25], 'BackgroundColor', [0.12 0.15 0.12], ...
                 'ForegroundColor', [1 0.6 0.6], 'FontSize', 11);
h.cam_pitch_lbl = uicontrol(cam_panel, 'Style', 'text', 'String', 'P: 0.0', ...
                  'Position', [100 60 80 25], 'BackgroundColor', [0.12 0.15 0.12], ...
                  'ForegroundColor', [0.6 1 0.6], 'FontSize', 11);
h.cam_yaw_lbl = uicontrol(cam_panel, 'Style', 'text', 'String', 'Y: 0.0', ...
                'Position', [190 60 80 25], 'BackgroundColor', [0.12 0.15 0.12], ...
                'ForegroundColor', [0.6 0.6 1], 'FontSize', 11);

h.enc_lbl = uicontrol(cam_panel, 'Style', 'text', 'String', 'Encoder: R:0 P:0 Y:0', ...
            'Position', [10 25 260 22], 'BackgroundColor', [0.12 0.15 0.12], ...
            'ForegroundColor', [1 1 0.5], 'FontSize', 9);

%% DRIFT PANEL
drift_panel = uipanel('Title', 'DRIFT INJECTION', 'Position', [0.02 0.45 0.45 0.28], ...
```

```
                'BackgroundColor', [0.2 0.1 0.1], 'ForegroundColor', [1 0.5 0.5], 'FontSize', 10);

h.drift_check = uicontrol(drift_panel, 'Style', 'checkbox', 'String', 'Enable Drift', ...
                'Position', [10 140 100 25], 'Value', 0, ...
                'BackgroundColor', [0.2 0.1 0.1], 'ForegroundColor', [1 0.8 0.8], ...
                'Callback', @(src,~) setDriftEnabled(fig, src));

% Roll drift
uicontrol(drift_panel, 'Style', 'text', 'String', 'Roll (deg/s):', ...
        'Position', [10 110 80 20], 'BackgroundColor', [0.2 0.1 0.1], ...
        'ForegroundColor', 'white', 'HorizontalAlignment', 'left');
h.drift_roll_slider = uicontrol(drift_panel, 'Style', 'slider', ...
                'Position', [100 113 150 18], 'Min', -5, 'Max', 5, 'Value', 0, ...
                'Callback', @(src,~) setDriftRate(fig, 1, src));
h.drift_roll_lbl = uicontrol(drift_panel, 'Style', 'text', 'String', '0.00', ...
                'Position', [255 110 45 20], 'BackgroundColor', [0.2 0.1 0.1], ...
                'ForegroundColor', [1 0.6 0.6]);

% Pitch drift
uicontrol(drift_panel, 'Style', 'text', 'String', 'Pitch (deg/s):', ...
        'Position', [10 80 80 20], 'BackgroundColor', [0.2 0.1 0.1], ...
        'ForegroundColor', 'white', 'HorizontalAlignment', 'left');
h.drift_pitch_slider = uicontrol(drift_panel, 'Style', 'slider', ...
                'Position', [100 83 150 18], 'Min', -5, 'Max', 5, 'Value', 0, ...
```

```
                        'Callback', @(src,~) setDriftRate(fig, 2, src));
h.drift_pitch_lbl = uicontrol(drift_panel, 'Style', 'text', 'String', '0.00', ...
                        'Position', [255 80 45 20], 'BackgroundColor', [0.2 0.1 0.1], ...
                        'ForegroundColor', [0.6 1 0.6]);

% Yaw drift
uicontrol(drift_panel, 'Style', 'text', 'String', 'Yaw (deg/s):', ...
        'Position', [10 50 80 20], 'BackgroundColor', [0.2 0.1 0.1], ...
        'ForegroundColor', 'white', 'HorizontalAlignment', 'left');
h.drift_yaw_slider = uicontrol(drift_panel, 'Style', 'slider', ...
                        'Position', [100 53 150 18], 'Min', -5, 'Max', 5, 'Value', 0, ...
                        'Callback', @(src,~) setDriftRate(fig, 3, src));
h.drift_yaw_lbl = uicontrol(drift_panel, 'Style', 'text', 'String', '0.00', ...
                        'Position', [255 50 45 20], 'BackgroundColor', [0.2 0.1 0.1], ...
                        'ForegroundColor', [0.6 0.6 1]);

uicontrol(drift_panel, 'Style', 'pushbutton', 'String', 'Reset Drift', ...
        'Position', [10 15 100 28], 'Callback', @(~,~) resetDrift(fig));

h.drift_accum_lbl = uicontrol(drift_panel, 'Style', 'text', ...
                        'String', 'Accum: R:0 P:0 Y:0', ...
                        'Position', [120 15 180 22], 'BackgroundColor', [0.2 0.1 0.1], ...
                        'ForegroundColor', [1 0.8 0.5]);
```

```
%% STANDALONE WAVEFORM PANEL
wave_panel = uipanel('Title', 'Standalone Waveform (when not following)', ...
                'Position', [0.49 0.45 0.5 0.28], ...
                'BackgroundColor', [0.1 0.15 0.2], 'ForegroundColor', [0.5 0.8 1], 'FontSize', 10);

% X axis
h.wave_x_check = uicontrol(wave_panel, 'Style', 'checkbox', 'String', 'X (Roll)', ...
                    'Position', [10 140 80 20], 'Value', 0, ...
                    'BackgroundColor', [0.1 0.15 0.2], 'ForegroundColor', [1 0.5 0.5], ...
                    'Callback', @(src,~) setWaveEnabled(fig, 1, src));
uicontrol(wave_panel, 'Style', 'text', 'String', 'Amp:', 'Position', [100 140 35 18], ...
        'BackgroundColor', [0.1 0.15 0.2], 'ForegroundColor', 'white');
h.wave_x_amp = uicontrol(wave_panel, 'Style', 'edit', 'String', '30', ...
                   'Position', [140 140 40 20], 'Callback', @(src,~) setWaveAmp(fig, 1, src));
uicontrol(wave_panel, 'Style', 'text', 'String', 'Freq:', 'Position', [190 140 35 18], ...
        'BackgroundColor', [0.1 0.15 0.2], 'ForegroundColor', 'white');
h.wave_x_freq = uicontrol(wave_panel, 'Style', 'edit', 'String', '0.3', ...
                    'Position', [230 140 40 20], 'Callback', @(src,~) setWaveFreq(fig, 1, src));

% Y axis
h.wave_y_check = uicontrol(wave_panel, 'Style', 'checkbox', 'String', 'Y (Pitch)', ...
                    'Position', [10 110 80 20], 'Value', 0, ...
                    'BackgroundColor', [0.1 0.15 0.2], 'ForegroundColor', [0.5 1 0.5], ...
                    'Callback', @(src,~) setWaveEnabled(fig, 2, src));
```

```
uicontrol(wave_panel, 'Style', 'text', 'String', 'Amp:', 'Position', [100 110 35 18], ...
        'BackgroundColor', [0.1 0.15 0.2], 'ForegroundColor', 'white');
h.wave_y_amp = uicontrol(wave_panel, 'Style', 'edit', 'String', '30', ...
                  'Position', [140 110 40 20], 'Callback', @(src,~) setWaveAmp(fig, 2, src));
uicontrol(wave_panel, 'Style', 'text', 'String', 'Freq:', 'Position', [190 110 35 18], ...
        'BackgroundColor', [0.1 0.15 0.2], 'ForegroundColor', 'white');
h.wave_y_freq = uicontrol(wave_panel, 'Style', 'edit', 'String', '0.2', ...
                   'Position', [230 110 40 20], 'Callback', @(src,~) setWaveFreq(fig, 2, src));

% Z axis
h.wave_z_check = uicontrol(wave_panel, 'Style', 'checkbox', 'String', 'Z (Yaw)', ...
                    'Position', [10 80 80 20], 'Value', 0, ...
                    'BackgroundColor', [0.1 0.15 0.2], 'ForegroundColor', [0.5 0.5 1], ...
                    'Callback', @(src,~) setWaveEnabled(fig, 3, src));
uicontrol(wave_panel, 'Style', 'text', 'String', 'Amp:', 'Position', [100 80 35 18], ...
        'BackgroundColor', [0.1 0.15 0.2], 'ForegroundColor', 'white');
h.wave_z_amp = uicontrol(wave_panel, 'Style', 'edit', 'String', '30', ...
                  'Position', [140 80 40 20], 'Callback', @(src,~) setWaveAmp(fig, 3, src));
uicontrol(wave_panel, 'Style', 'text', 'String', 'Freq:', 'Position', [190 80 35 18], ...
        'BackgroundColor', [0.1 0.15 0.2], 'ForegroundColor', 'white');
h.wave_z_freq = uicontrol(wave_panel, 'Style', 'edit', 'String', '0.1', ...
                   'Position', [230 80 40 20], 'Callback', @(src,~) setWaveFreq(fig, 3, src));

% Noise
```

```
h.noise_check = uicontrol(wave_panel, 'Style', 'checkbox', 'String', 'Add Noise', ...
                    'Position', [10 45 80 20], 'Value', 0, ...
                    'BackgroundColor', [0.1 0.15 0.2], 'ForegroundColor', 'white', ...
                    'Callback', @(src,~) setNoiseEnabled(fig, src));
uicontrol(wave_panel, 'Style', 'text', 'String', 'Amp:', 'Position', [100 45 35 18], ...
        'BackgroundColor', [0.1 0.15 0.2], 'ForegroundColor', 'white');
h.noise_amp_edit = uicontrol(wave_panel, 'Style', 'edit', 'String', '0.5', ...
                      'Position', [140 45 40 20], 'Callback', @(src,~) setNoiseAmp(fig, src));

%% CONTROL BUTTONS
ctrl_panel = uipanel('Title', 'Control', 'Position', [0.66 0.75 0.32 0.18], ...
                'BackgroundColor', [0.15 0.15 0.2], 'ForegroundColor', 'white', 'FontSize', 10);

h.pause_btn = uicontrol(ctrl_panel, 'Style', 'togglebutton', 'String', 'PAUSE', ...
                    'Position', [10 60 80 35], 'FontSize', 10, ...
                    'Callback', @(src,~) togglePause(fig, src));

uicontrol(ctrl_panel, 'Style', 'pushbutton', 'String', 'STOP', ...
        'Position', [100 60 80 35], 'BackgroundColor', [0.8 0.2 0.2], ...
        'ForegroundColor', 'white', 'FontWeight', 'bold', ...
        'Callback', @(~,~) close(fig));

h.status_lbl = uicontrol(ctrl_panel, 'Style', 'text', 'String', 'Packets: 0', ...
                    'Position', [10 20 170 25], 'BackgroundColor', [0.15 0.15 0.2], ...
```

```
                'ForegroundColor', [0.7 0.7 0.7]);


%% PLOT
h.ax = axes('Parent', fig, 'Position', [0.08 0.08 0.88 0.32], ...
        'Color', [0.05 0.05 0.1], 'XColor', [0.7 0.7 0.7], 'YColor', [0.7 0.7 0.7]);
title(h.ax, 'Encoder Output (sent to Panel)', 'Color', 'white');
xlabel(h.ax, 'Time (s)', 'Color', [0.7 0.7 0.7]);
grid(h.ax, 'on'); h.ax.GridColor = [0.3 0.3 0.3];
hold(h.ax, 'on');
h.plot_roll = plot(h.ax, 0, 0, 'r-', 'LineWidth', 1.5);
h.plot_pitch = plot(h.ax, 0, 0, 'g-', 'LineWidth', 1.5);
h.plot_yaw = plot(h.ax, 0, 0, 'b-', 'LineWidth', 1.5);
legend(h.ax, {'Roll', 'Pitch', 'Yaw'}, 'TextColor', 'white', 'Location', 'northeast');
ylim(h.ax, [-360, 360]);


% Buffers
BUF_SIZE = 300;
h.t_buf = zeros(1, BUF_SIZE);
h.r_buf = zeros(1, BUF_SIZE);
h.p_buf = zeros(1, BUF_SIZE);
h.y_buf = zeros(1, BUF_SIZE);
h.buf_idx = 1;


guidata(fig, h);
```

```
    %% TIMER
    h.timer = timer('ExecutionMode', 'fixedRate', 'Period', DT, ...
                    'TimerFcn', @(~,~) updateSim(fig));
    start(h.timer);
    guidata(fig, h);

    fprintf('[OK] Simulation running\n');
end

%% CALLBACKS
function setMode(fig, follow)
    h = guidata(fig);
    h.follow_mode = follow;
    set(h.mode_follow, 'Value', follow);
    set(h.mode_standalone, 'Value', ~follow);
    guidata(fig, h);
end

function setDriftEnabled(fig, src)
    h = guidata(fig);
    h.drift_enabled = get(src, 'Value');
    guidata(fig, h);
end
```

```
function setDriftRate(fig, axis, src)
    h = guidata(fig);
    val = get(src, 'Value');
    h.drift_rate(axis) = val;
    lbls = {h.drift_roll_lbl, h.drift_pitch_lbl, h.drift_yaw_lbl};
    set(lbls{axis}, 'String', sprintf('%.2f', val));
    guidata(fig, h);
end

function resetDrift(fig)
    h = guidata(fig);
    h.accum_drift = [0, 0, 0];
    guidata(fig, h);
end

function setWaveEnabled(fig, axis, src)
    h = guidata(fig);
    h.wave_enabled(axis) = get(src, 'Value');
    guidata(fig, h);
end

function setWaveAmp(fig, axis, src)
    h = guidata(fig);
```

```
    val = str2double(get(src, 'String'));
    if ~isnan(val), h.wave_amp(axis) = val; end
    guidata(fig, h);
end

function setWaveFreq(fig, axis, src)
    h = guidata(fig);
    val = str2double(get(src, 'String'));
    if ~isnan(val) && val > 0, h.wave_freq(axis) = val; end
    guidata(fig, h);
end

function setNoiseEnabled(fig, src)
    h = guidata(fig);
    h.noise_enabled = get(src, 'Value');
    guidata(fig, h);
end

function setNoiseAmp(fig, src)
    h = guidata(fig);
    val = str2double(get(src, 'String'));
    if ~isnan(val), h.noise_amp = val; end
    guidata(fig, h);
end
```

```
function togglePause(fig, src)
    h = guidata(fig);
    h.paused = get(src, 'Value');
    if h.paused, set(src, 'String', 'RESUME');
    else, set(src, 'String', 'PAUSE'); end
    guidata(fig, h);
end

function onClose(fig, ~)
    h = guidata(fig);
    h.running = false;
    if isfield(h, 'timer') && isvalid(h.timer)
        stop(h.timer); delete(h.timer);
    end
    try fclose(h.cam_socket); delete(h.cam_socket); catch, end
    try fclose(h.enc_socket); delete(h.enc_socket); catch, end
    delete(fig);
    fprintf('[OK] Simulation stopped\n');
end

%% MAIN UPDATE
function updateSim(fig)
    if ~isvalid(fig), return; end
```

```
h = guidata(fig);
if ~h.running || h.paused, return; end

DT = 1/30;
h.t = h.t + DT;

% Read camera data
try
   while h.cam_socket.BytesAvailable > 0
      data = fread(h.cam_socket, h.cam_socket.BytesAvailable, 'char');
      json_str = char(data');
      % Parse JSON manually (MATLAB 2018 compatible)
      r_match = regexp(json_str, '"roll":\s*([-\d.]+)', 'tokens');
      p_match = regexp(json_str, '"pitch":\s*([-\d.]+)', 'tokens');
      y_match = regexp(json_str, '"yaw":\s*([-\d.]+)', 'tokens');
      if ~isempty(r_match), h.cam_roll = str2double(r_match{1}{1}); end
      if ~isempty(p_match), h.cam_pitch = str2double(p_match{1}{1}); end
      if ~isempty(y_match), h.cam_yaw = str2double(y_match{1}{1}); end
      h.last_cam_time = now;
      h.cam_connected = true;
   end
catch
end
```

```
% Check camera timeout (2 seconds)
if (now - h.last_cam_time) * 86400 > 2
   h.cam_connected = false;
end

% Calculate encoder output
if h.follow_mode && h.cam_connected
   % Follow camera + drift
   enc_roll = h.cam_roll;
   enc_pitch = h.cam_pitch;
   enc_yaw = h.cam_yaw;
else
   % Standalone waveform
   enc_roll = 0; enc_pitch = 0; enc_yaw = 0;
   if h.wave_enabled(1)
      enc_roll = h.wave_amp(1) * sin(2*pi*h.wave_freq(1)*h.t);
   end
   if h.wave_enabled(2)
      enc_pitch = h.wave_amp(2) * sin(2*pi*h.wave_freq(2)*h.t);
   end
   if h.wave_enabled(3)
      enc_yaw = h.wave_amp(3) * sin(2*pi*h.wave_freq(3)*h.t);
   end
end
```

```
% Add drift
if h.drift_enabled
   h.accum_drift = h.accum_drift + h.drift_rate * DT;
end
enc_roll = enc_roll + h.accum_drift(1);
enc_pitch = enc_pitch + h.accum_drift(2);
enc_yaw = enc_yaw + h.accum_drift(3);

% Add noise
if h.noise_enabled
   enc_roll = enc_roll + h.noise_amp * randn();
   enc_pitch = enc_pitch + h.noise_amp * randn();
   enc_yaw = enc_yaw + h.noise_amp * randn();
end

% Send encoder data as JSON
json_str = sprintf('{"roll":%.2f,"pitch":%.2f,"yaw":%.2f}', enc_roll, enc_pitch, enc_yaw);
try
   fwrite(h.enc_socket, json_str, 'char');
   h.packet_count = h.packet_count + 1;
catch
end
```

```
% Update buffers
idx = h.buf_idx;
h.t_buf(idx) = h.t;
h.r_buf(idx) = enc_roll;
h.p_buf(idx) = enc_pitch;
h.y_buf(idx) = enc_yaw;
h.buf_idx = mod(idx, length(h.t_buf)) + 1;

% Update plots (every 5 frames)
if mod(h.packet_count, 5) == 0
    valid = h.t_buf > 0;
    set(h.plot_roll, 'XData', h.t_buf(valid), 'YData', h.r_buf(valid));
    set(h.plot_pitch, 'XData', h.t_buf(valid), 'YData', h.p_buf(valid));
    set(h.plot_yaw, 'XData', h.t_buf(valid), 'YData', h.y_buf(valid));
    if h.t > 1
        xlim(h.ax, [max(0, h.t-10), h.t]);
    end
end

% Update UI (every 10 frames)
if mod(h.packet_count, 10) == 0
    % Camera status
    if h.cam_connected
        set(h.cam_status, 'String', 'Camera: Connected', 'ForegroundColor', [0.5 1 0.5]);
```

```
        else
            set(h.cam_status, 'String', 'Camera: Waiting...', 'ForegroundColor', [1 0.5 0.5]);
        end

        % Camera values
        set(h.cam_roll_lbl, 'String', sprintf('R: %.1f', h.cam_roll));
        set(h.cam_pitch_lbl, 'String', sprintf('P: %.1f', h.cam_pitch));
        set(h.cam_yaw_lbl, 'String', sprintf('Y: %.1f', h.cam_yaw));

        % Encoder output
        set(h.enc_lbl, 'String', sprintf('Encoder: R:%.1f P:%.1f Y:%.1f', enc_roll, enc_pitch, enc_yaw));

        % Drift accumulator
        set(h.drift_accum_lbl, 'String', sprintf('Accum: R:%.1f P:%.1f Y:%.1f', ...
            h.accum_drift(1), h.accum_drift(2), h.accum_drift(3)));

        % Packets
        set(h.status_lbl, 'String', sprintf('Packets: %d', h.packet_count));
    end

    guidata(fig, h);
    drawnow limitrate;
end
```

E. motion_blur_test.py

```
#!/usr/bin/env python3
"""
Motion Blur Test V2 - Simplified
--------------------------------
Daha basit versiyon - tuş basımlarını debug ediyor
"""

import cv2
import numpy as np
import subprocess
import time

CAPTURE_WIDTH = 1280
CAPTURE_HEIGHT = 800
FRAMERATE = 30

ROI_X = 186
ROI_Y = 0
ROI_W = 860
ROI_H = 799

ARUCO_DICT_TYPE = cv2.aruco.DICT_5X5_50

# Global camera process
```

```
cam_proc = None
current_shutter = 0
current_gain = 0



def start_camera(shutter, gain):
    global cam_proc

    if cam_proc is not None:
        cam_proc.terminate()
        cam_proc.wait()
        time.sleep(0.5)

    cmd = [
        "rpicam-vid",
        "--width", str(CAPTURE_WIDTH),
        "--height", str(CAPTURE_HEIGHT),
        "--framerate", str(FRAMERATE),
        "--timeout", "0",
        "--shutter", str(shutter),
        "--gain", str(gain),
        "--nopreview",
        "-o", "-",
        "--codec", "yuv420",
```

```
    ]
    print(f"[CAMERA] Starting: shutter={shutter}, gain={gain}")
    cam_proc = subprocess.Popen(cmd, stdout=subprocess.PIPE, stderr=subprocess.DEVNULL)
    time.sleep(0.5)
    return cam_proc


def read_frame():
    global cam_proc
    if cam_proc is None:
        return None
    frame_size = CAPTURE_WIDTH * CAPTURE_HEIGHT * 3 // 2
    raw = cam_proc.stdout.read(frame_size)
    if len(raw) != frame_size:
        return None
    yuv = np.frombuffer(raw, dtype=np.uint8).reshape((CAPTURE_HEIGHT * 3 // 2, CAPTURE_WIDTH))
    bgr = cv2.cvtColor(yuv, cv2.COLOR_YUV2BGR_I420)
    return bgr


def setup_aruco():
    dictionary = cv2.aruco.Dictionary_get(ARUCO_DICT_TYPE)
    params = cv2.aruco.DetectorParameters_create()
    params.adaptiveThreshWinSizeMin = 3
```

```
    params.adaptiveThreshWinSizeMax = 41
    params.adaptiveThreshWinSizeStep = 6
    params.adaptiveThreshConstant = 7
    params.minMarkerPerimeterRate = 0.008
    params.maxMarkerPerimeterRate = 4.0
    params.polygonalApproxAccuracyRate = 0.06
    return dictionary, params


def main():
    global current_shutter, current_gain, cam_proc

    print("=" * 50)
    print("MOTION BLUR TEST V2")
    print("=" * 50)
    print("Tuşlar:")
    print("  1 = Shutter 0 (auto)")
    print("  2 = Shutter 1000 (1ms)")
    print("  3 = Shutter 2000 (2ms)")
    print("  4 = Shutter 3000 (3ms)")
    print("  5 = Shutter 5000 (5ms)")
    print("  6 = Gain 0 (auto)")
    print("  7 = Gain 4")
    print("  8 = Gain 8")
```

```
print("  9 = Gain 12")

print("  0 = Gain 16")

print("  q = Çık")

print("=" * 50)


# Shutter ve gain presetleri

shutter_presets = {

   ord('1'): 0,

   ord('2'): 1000,

   ord('3'): 2000,

   ord('4'): 3000,

   ord('5'): 5000,

}


gain_presets = {

   ord('6'): 0,

   ord('7'): 4,

   ord('8'): 8,

   ord('9'): 12,

   ord('0'): 16,

}


current_shutter = 0

current_gain = 0
```

```
start_camera(current_shutter, current_gain)

dictionary, params = setup_aruco()


fps_cnt = 0

fps_t = time.time()

fps = 0.0


try:

   while True:

      frame = read_frame()

      if frame is None:

         continue


      # FPS

      fps_cnt += 1

      dt = time.time() - fps_t

      if dt >= 1.0:

         fps = fps_cnt / dt

         fps_cnt = 0

         fps_t = time.time()


      # Detect markers

      x, y, w, h = ROI_X, ROI_Y, ROI_W, ROI_H
```

```
roi = frame[y:y+h, x:x+w]

gray = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)

corners, ids, _ = cv2.aruco.detectMarkers(gray, dictionary, parameters=params)


num_detected = len(ids) if ids is not None else 0


# Sharpness

laplacian_var = cv2.Laplacian(gray, cv2.CV_64F).var()


# Draw

vis = frame.copy()

cv2.rectangle(vis, (ROI_X, ROI_Y), (ROI_X + ROI_W, ROI_Y + ROI_H), (0, 255, 255), 2)


if corners and ids is not None:

   corners_offset = [c + np.array([ROI_X, ROI_Y], dtype=np.float32) for c in corners]

   cv2.aruco.drawDetectedMarkers(vis, corners_offset, ids)


# Text

y0 = 30

shutter_txt = "AUTO" if current_shutter == 0 else f"{current_shutter}µs"

gain_txt = "AUTO" if current_gain == 0 else str(current_gain)


cv2.putText(vis, f"SHUTTER: {shutter_txt} (keys 1-5)", (10, y0),

        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)
```

```
        y0 += 28
        cv2.putText(vis, f"GAIN: {gain_txt} (keys 6-0)", (10, y0),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)
        y0 += 35

        cv2.putText(vis, f"FPS: {fps:.1f}", (10, y0),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
        y0 += 28
        cv2.putText(vis, f"Detected: {num_detected}", (10, y0),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
        y0 += 28

        sharp_color = (0, 255, 0) if laplacian_var > 500 else (0, 165, 255) if laplacian_var > 200 else (0, 0,
255)
        cv2.putText(vis, f"Sharpness: {laplacian_var:.0f}", (10, y0),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, sharp_color, 2)

        cv2.imshow("Motion Blur Test V2", vis)

        key = cv2.waitKey(1) & 0xFF

        if key == ord('q'):
            print("[QUIT]")
            break
```

```
        # Shutter değiştir
        if key in shutter_presets:
            new_shutter = shutter_presets[key]
            if new_shutter != current_shutter:
                current_shutter = new_shutter
                print(f"[KEY] Shutter -> {current_shutter}")
                start_camera(current_shutter, current_gain)

        # Gain değiştir
        if key in gain_presets:
            new_gain = gain_presets[key]
            if new_gain != current_gain:
                current_gain = new_gain
                print(f"[KEY] Gain -> {current_gain}")
                start_camera(current_shutter, current_gain)

        # Debug: herhangi bir tuşa basıldığında göster
        if key != 255:
            print(f"[DEBUG] Key pressed: {key} (char: {chr(key) if 32 <= key < 127 else '?'})")

finally:
    if cam_proc:
        cam_proc.terminate()
```

```
        cam_proc.wait()
    cv2.destroyAllWindows()
    print(f"\n[SONUÇ] En son ayarlar:")
    print(f"  SHUTTER = {current_shutter}")
    print(f"  GAIN = {current_gain}")


if __name__ == "__main__":
  main()
```

F. pick_center_roi.py

```
#!/usr/bin/env python3
import cv2
import numpy as np
import subprocess
import os
import time

# ======================
# Capture settings
# ======================
W, H = 1280, 800
OUT_IMG = "one.jpg"     # saved capture (used for picking)
CAM_TIMEOUT_MS = 200    # capture timeout
```

```
# ROI suggestion margin (px)
MARGIN = 20

points = []  # [center, edge]
img = None
img_vis = None

def capture_frame():
    """
    Captures a single 1280x800 frame using rpicam-still and writes OUT_IMG.
    """
    # Remove old file to avoid confusion
    if os.path.exists(OUT_IMG):
        os.remove(OUT_IMG)

    cmd = [
        "rpicam-still",
        "--width", str(W),
        "--height", str(H),
        "--timeout", str(CAM_TIMEOUT_MS),
        "--nopreview",
        "--encoding", "jpg",
        "-o", OUT_IMG
    ]
```

```
    print("Capturing frame:", " ".join(cmd))
    subprocess.run(cmd, check=True)

    if not os.path.exists(OUT_IMG):
        raise RuntimeError(f"Capture failed: {OUT_IMG} was not created.")

def print_results(cx, cy, r, x0, y0, w, h):
    print("\n=== USE THESE IN YOUR TRACKER ===")
    print(f"CAPTURE_WIDTH  = {W}")
    print(f"CAPTURE_HEIGHT = {H}")
    print(f"SPHERE_CENTER_PX_FALLBACK = ({cx}, {cy})")
    print(f"SPHERE_RADIUS_PX = {r}")
    print(f"ROI_X = {x0}")
    print(f"ROI_Y = {y0}")
    print(f"ROI_W = {w}")
    print(f"ROI_H = {h}")
    print("================================\n")

def on_mouse(event, x, y, flags, param):
    global points, img_vis

    if event != cv2.EVENT_LBUTTONDOWN:
        return
```

```
points.append((x, y))
print("Clicked:", (x, y))

# 1) Center picked
if len(points) == 1:
    cv2.circle(img_vis, points[0], 5, (0, 255, 0), -1)
    cv2.putText(img_vis, "CENTER", (x+10, y),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,255,0), 2)

# 2) Edge picked -> radius + ROI suggestion
if len(points) == 2:
    cx, cy = points[0]
    ex, ey = points[1]
    r = int(np.hypot(ex - cx, ey - cy))

    cv2.circle(img_vis, (cx, cy), r, (0, 255, 0), 2)

    # ROI suggestion
    x0 = max(0, cx - r - MARGIN)
    y0 = max(0, cy - r - MARGIN)
    x1 = min(img_vis.shape[1] - 1, cx + r + MARGIN)
    y1 = min(img_vis.shape[0] - 1, cy + r + MARGIN)

    cv2.rectangle(img_vis, (x0, y0), (x1, y1), (0, 0, 255), 2)
```

```
        cv2.putText(img_vis, "ROI", (x0+10, y0+30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0,0,255), 2)

        print_results(cx, cy, r, x0, y0, x1-x0, y1-y0)

        print("Tip: press 'r' to reset points, or 'q' to quit.")

def main():
    global img, img_vis, points

    # 1) Capture
    capture_frame()

    # 2) Load captured image
    img = cv2.imread(OUT_IMG)
    if img is None:
        raise RuntimeError(f"{OUT_IMG} could not be read. Capture may have failed.")

    if img.shape[1] != W or img.shape[0] != H:
        print(f"[WARN] Image size is {img.shape[1]}x{img.shape[0]} but expected {W}x{H}.")

    img_vis = img.copy()

    # 3) UI
```

```
cv2.namedWindow("Pick Sphere Center & Edge", cv2.WINDOW_NORMAL)
cv2.resizeWindow("Pick Sphere Center & Edge", 1000, 650)
cv2.setMouseCallback("Pick Sphere Center & Edge", on_mouse)

print("1) Click the TRUE sphere center (pixel).")
print("2) Then click a point on the sphere edge (circle boundary).")
print("Keys: 'r' reset, 'c' recapture, 'q' quit.")

while True:
    cv2.imshow("Pick Sphere Center & Edge", img_vis)
    k = cv2.waitKey(30) & 0xFF

    if k == ord('q'):
        break

    # Reset picks
    if k == ord('r'):
        points = []
        img_vis = img.copy()
        print("[RESET] Click center then edge again.")

    # Re-capture a fresh frame (useful if lighting changed)
    if k == ord('c'):
        print("[RECAPTURE] Taking a new image...")
```

```
            capture_frame()
            img = cv2.imread(OUT_IMG)
            if img is None:
                print("[ERROR] Could not read recaptured image.")
                continue
            points = []
            img_vis = img.copy()
            print("[OK] New frame loaded. Click center then edge again.")

    cv2.destroyAllWindows()

if __name__ == "__main__":
    main()
```

G. calibrate_chessboard.py

```
#!/usr/bin/env python3
import glob
import json
import cv2
import numpy as np

# === Bu board için inner corners ===
# Senin en son verdiğim A4 landscape board: inner 8x5
PATTERN_SIZE = (8, 5)
```

```
# Kare boyu: sen "29 mm civarı geliyor" dedin.
# Kalibrasyonun tutarlı olması için burada 0.029 m kullanıyoruz.
SQUARE_SIZE_M = 0.0192  # metre

# Corner refinement
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 1e-6)

# 3D object points (0,0,0), (1,0,0), ... * square_size
objp = np.zeros((PATTERN_SIZE[0] * PATTERN_SIZE[1], 3), np.float32)
objp[:, :2] = np.mgrid[0:PATTERN_SIZE[0], 0:PATTERN_SIZE[1]].T.reshape(-1, 2)
objp *= SQUARE_SIZE_M

objpoints = []
imgpoints = []

images = sorted(glob.glob("calib_imgs/*.png"))
if len(images) < 10:
    raise RuntimeError("calib_imgs/ içinde en az 10 görüntü olmalı (tercihen 20-30).")

img_size = None
used = 0

for fname in images:
    img = cv2.imread(fname, cv2.IMREAD_GRAYSCALE)
```

```
    if img is None:
        continue

    if img_size is None:
        img_size = (img.shape[1], img.shape[0])  # (W,H)

    ret, corners = cv2.findChessboardCorners(
        img, PATTERN_SIZE,
        flags=cv2.CALIB_CB_ADAPTIVE_THRESH + cv2.CALIB_CB_NORMALIZE_IMAGE
    )

    if ret:
        corners2 = cv2.cornerSubPix(img, corners, (11, 11), (-1, -1), criteria)
        objpoints.append(objp)
        imgpoints.append(corners2)
        used += 1

print("Total images:", len(images))
print("Used images:", used)
if used < 8:
    raise RuntimeError("Yeterli iyi görüntü yok. Board'u daha net/farklı açılarla çek.")

# Calibrate
ret, K, dist, rvecs, tvecs = cv2.calibrateCamera(
```

```
    objpoints, imgpoints, img_size, None, None
)

print("Reprojection error:", ret)
print("K:\n", K)
print("dist:\n", dist.ravel())

out = {
    "imageSize": {"width": img_size[0], "height": img_size[1]},
    "patternSize": {"inner_x": PATTERN_SIZE[0], "inner_y": PATTERN_SIZE[1]},
    "squareSize_m": SQUARE_SIZE_M,
    "K": K.tolist(),
    "dist": dist.ravel().tolist(),
    "reprojection_error": float(ret),
    "used_images": int(used),
    "total_images": int(len(images)),
}
with open("camera_calib_chessboard.json", "w") as f:
    json.dump(out, f, indent=2)

print("Saved: camera_calib_chessboard.json")
```

10. REFERENCES

1. ***Arducam.***
   USB Camera Module Documentation.
   Arducam Official Documentation.

2. ***Bradski, G.***
   The OpenCV Library.
   Dr. Dobb's Journal of Software Tools, 2000.

3. ***Craig, J. J.***
   Introduction to Robotics: Mechanics and Control.
   Pearson, 3rd Edition, 2004.

4. ***Gonzalez, R. C., & Woods, R. E.***
   Digital Image Processing.
   Pearson Education, 4th Edition, 2018.

5. ***Hartley, R., & Zisserman, A.***
   Multiple View Geometry in Computer Vision.
   Cambridge University Press, 2nd Edition, 2004.

6. ***Hunter, J. D.***
   Matplotlib: A 2D Graphics Environment.
   Computing in Science & Engineering, 2007.

7. ***Jolliffe, I. T.***
   Principal Component Analysis.
   Springer Series in Statistics, 2nd Edition, 2002.

8. ***Logitech.***
Logitech C920 / C922 Webcam Specifications.
Logitech Official Documentation.

9. ***OpenCV Documentation.***
OpenCV 4.x Documentation.
https://docs.opencv.org

10. ***PixArt Imaging Inc.***
ADNS-2610 Optical Mouse Sensor Datasheet.
PixArt Imaging, 2005.

11. ***PixArt Imaging Inc.***
ADNS-3080 High Performance Optical Mouse Sensor Datasheet.
PixArt Imaging.

12. ***Siciliano, B., Sciavicco, L., Villani, L., & Oriolo, G.***
Robotics: Modelling, Planning and Control.
Springer, 2009.

13. ***Van der Walt, S., Colbert, S. C., & Varoquaux, G.***
The NumPy Array: A Structure for Efficient Numerical Computation.
Computing in Science & Engineering, 2011.